Transparent Arithmetic over the Combinators K and S: Divisibility Automata, Factoring as Rewriting, and the Search for a Prime-Revealing Representation

Working paper v0.9, Project secret2, August 11, 2026. Appendix C synchronized against the shared proof module at 3,337 lines.

Abstract

We search for a combinator representation of the natural numbers whose terms make primality testing and factor extraction immediate. Three external facts bound the search before any design work. A conversion barrier: a representation that is compact, cheaply encoded, cheaply decoded, and cheaply factored would make integer factorization polynomial-time. An asymmetry: primality testing is unconditionally polynomial-time, so only extraction is barred by complexity assumptions. A locality obstruction: the primes are not a regular language, and no infinite subset of them is context-free, in any base, so no positional encoding can carry a bounded-window primality marker. We therefore relativize the goal: a representation is transparent for a property when its canonical normal form under a chosen equality displays the property in a bounded root window, and the cost moves into normalization. The paper builds the machinery and measures it. Binary numerals from doubling and successor make parity root-local and divisibility by 2^j local at depth j, while remaining logarithmically compact as term DAGs. Divisibility by any fixed k becomes a family of three local rewrite rules whose probes cost exactly the binary length of n; the prime 8,191 is certified in 1,157 local steps. F_arith, a rewrite system whose normal forms are factored forms, makes primality literally irreducibility and unique factorization literally confluence, with termination provable, in contrast to the multiplying orientation of the predecessor paper. Replacing its unary certificates by rewrite division on binary leaves collapses certification from linear to logarithmic: 1,001 drops from 1,182 to 18 certification steps, and 65,535 is factored with full certificates in 344 local steps. What remains is exactly one cost, the ascending search for divisors, and it sits where the barrier says it must. The residue bundles predicted by the separation and fusion reading of K and S are then built: an inert constant-size carrier per prime basis on which divisibility by every basis prime is a window read, primality on the validity window is a conjunction of window reads, and addition and multiplication cost one local rewrite per coordinate regardless of the operands, with aliasing and hard comparison arriving exactly where the barrier demands. Finally, an exhaustive staged search over all 278,707,802 closed weak normal forms up to fifteen leaves, together with all raw terms up to nine, maps the small iterator world: 33 distinct orbit signatures, among them doubling at nine leaves, the honest 2n + 1, tripling, and the exponential 2 to the n at twelve, quadrupling and 2 n squared at fourteen, and n(2n + 1) at fifteen. None of 781 alphabets over these actions separates primes from composites by cheap reduction observables; the staged screen promoted and then executed two finite-window artifacts along the way. The grail, if it exists, is not below fifteen leaves.

1. Introduction

The predecessor paper [1] built a number system on {K, S} trees and characterized primality twice: as universal indecomposability under grafting, and as unreachability of product boxes under an in-

verse rewriting system. Both characterizations re-express trial division, and the unary comb encoding makes every divisibility probe cost linear in n. This paper raises the bar. We want a representation of the natural numbers inside combinatory logic in which testing primality and extracting factors is immediate: visible in the structure of the term, at cost far below re-deriving it.

The paper proceeds in three movements. Sections 2 and 3 fix what 'immediate' can coherently mean, against three external facts, and specify the properties any candidate must balance. Sections 4 to 6 build and measure the positive machinery: binary numerals whose divisibility is automatic, and a factoring rewrite system whose normal forms are factored forms, first with linear certificates and then with logarithmic ones. Sections 7 and 8 look outward: a reading of K and S that predicts the next representation, and an exhaustive search over all small representations that maps how much of the design space the small terms actually contain.

Every quantitative claim in this paper is machine-checked. The experiments are consolidated in a single Idris2 module, reproduced verbatim in Appendix B, and every reported figure is printed by its run. Beyond that discipline, the strongest schematic claims of the paper and of [1] are now theorems in the type-theoretic sense: a companion proof module, total and free of postulates, is reported in Section 9 and reproduced verbatim in Appendix C.

2. What 'immediate' can and cannot mean

Three facts bound the search space before any design work starts.

2.1 The conversion barrier

Observation 1. Suppose a representation R comes with: encoding from standard binary in time polynomial in log n; decoding back in time polynomial in the representation size; representation size polynomial in log n; and a factor-extraction operation that, for composite n, produces representations of some a, b with $n = a \cdot b$ and a, b at least 2, in time polynomial in the representation size. Then integer factorization is polynomial-time.

Proof. Encode, extract, decode, and recurse on both parts; the recursion depth is at most log n, and every level costs polynomial time in log n. ∎

No such algorithm is known, and much of cryptography wagers that none exists. So every honest design drops at least one leg, and the choice of which leg to drop is the design decision:

- Drop cheap encoding: the representation is the factored form itself. Entry costs a factorization; afterward multiplication, divisibility, primality, and factor extraction are literally free, while addition becomes as hard as factoring the sum.
- Drop compactness: unary. The predecessor paper lives here; its full primality certificates cost order n squared steps, exponential in the input length, exactly as the barrier predicts.
- Drop full extraction: residue systems expose only factors below a smoothness bound; everything above it is invisible.
- Drop nothing, but relativize: make transparency a property of canonical forms under an equality whose normalization carries the cost. This is the move native to this project, and Section 2.4 turns it into the working definition.

2.2 Testing is not extraction

Primality testing is in polynomial time unconditionally [7]. A compact representation whose canonical forms wear a structural primality marker therefore contradicts no complexity assumption; only factor extraction is barred by Observation 1. The two goals separate, and the specification of Section 3 keeps them separate.

2.3 The locality obstruction for positional encodings

For digit-style representations the obstruction to primality is not complexity but locality. The binary primes are not a regular language [4]; neither the primes nor any infinite subset of them is context-

free in any base [5]; and any finite automaton that is merely correct on inputs up to N needs a number of states polynomial in N [6]. Consequence: if R(n) is, up to renaming, the base-b digit word of n, then no bounded root window, no finite tree automaton, and no approximately correct small automaton decides primality on it. Structural primality forces the representation away from positional encodings altogether, toward factored forms, residue bundles, or something not yet on the map.

2.4 The working definition

Let =E be an equality on terms generated by a rewriting system with unique normal forms on the fragment of interest.

Definition 1. A representation R is **transparent for a property Q under =E** if the =E-normal form of R(n) decides Q within a root window of bounded depth, by a fixed pattern that does not grow with n.

The cost of Q then lives in normalization, not in the representation, and the research question becomes concrete: which equality has factored forms as its normal forms, how expensive is normalization, and where exactly does the residual cost hide? Under this definition the fundamental theorem of arithmetic itself becomes a rewriting statement: normal forms are factored forms, and their uniqueness is the confluence of the factoring system. The predecessor paper listed that formulation as open; Section 6 realizes it.

3. Specification and design space

3.1 The property schema

The schema doubles as the fitness function for the enumerative search of Section 8. For a candidate representation R with ambient equality =E:

- **P0 adequacy.** R is injective on values; zero or one and a finite constructor set are representable; canonical forms are constructible for every n.
- **P1 size.** Tree size and DAG size (distinct subterms) of R(n), as functions of n. Section 4 shows the two can differ exponentially.
- **P2 canonicity.** Each number has a unique =E-normal form, and =E is decidable on the fragment.
- **P3 arithmetic.** Combinators for successor, doubling, addition, and multiplication, with normalization cost bounds relative to representation size.
- **P4 order.** Zero test and comparison, with costs.
- **P5 divisibility transparency.** For each k, the window depth d(k) such that n mod k is a function of the depth-d(k) root window of the canonical form, or else the scan cost when no bounded window suffices.
- **P6 primality transparency.** A structural predicate, a fixed tree-automaton pattern on canonical forms, deciding primality. Carried annotations do not count; the shape itself must speak.
- **P7 factor extraction.** A rewrite or combinator that reaches representations of a nontrivial factor pair from R(n), with cost profile.
- **P8 conversion.** Encoding from and decoding to standard binary, with costs. Observation 1 links P1, P3, P7, and P8: they cannot all be cheap.
- **P9 closure.** Transparency (P5 to P7) is preserved by the operations of P3; a marker that dies after one addition is decoration, not structure.

3.2 The design space, mapped

Representation	Tree size	DAG size	Add	Mult	Div by 2^j	Div by k	Primality	Factor extraction	Encode

Unary combs [1]	n	n	concat	R_arith, n k steps	scan n	scan n	aperiodicity, scan	stride search, n^2	n steps
Church numerals	n	n	easy	easy	opaque	opaque	opaque	opaque	n steps
Binary, doubling and successor (Sec. 4)	n	log n	shift-add	shift-add	window depth j	digit scan, log n	blocked by 2.3	probes plus division, log per split (Sec. 6)	log n
Scott-style BinPos	log n	log n	programmed	programmed	window depth j	digit scan, log n	blocked by 2.3	open	log n
Factored forms	log n	log n	as hard as factoring	multiset union, free	free	free	free (singleton test)	free	factoring
Residue bundles (built, Sec. 7)	log n	log n	coordinate-wise	coordinate-wise	local per bundled prime	local if k bundled	smoothness only	small factors only	log n per prime
Hereditarily binary (Tarau) [10]	log n	log n	structural	structural	partial	opaque	opaque	opaque	log n

Reading the table against Observation 1: the factored-form row is the only one with free extraction, and it pays at the addition and encoding columns, exactly as the barrier demands. The residue row buys locality for every bundled prime and pays with partiality and with hard comparison. The positional rows are blocked at the primality column by Section 2.3 no matter how the arithmetic is tuned. The unary row is the control group: everything visible, everything expensive.

4. Binary numerals from doubling and successor

Inside combinatory logic, with B = S(KS)K and I = SKK as in [1], define

```
one    = SB(KI)                    the unary numeral 1
dbl m  = S(S(KB)m)m                the self-sum box, value 2m
suc m  = SB m                      the successor link, value m + 1
```

and let bin n follow the binary expansion of n: bin 1 = one; bin(2m) = dbl(bin m); bin(2m + 1) = suc(dbl(bin m)). The doubling box is the sum box of [1] with both summands equal; the least significant bit sits at the root, the leading one innermost. All of the following is machine-checked (Appendix B).

Adequacy. bin n and the unary numeral of n agree pointwise under weak reduction for n = 1 to 24: applied to fresh variables f and x, both reduce to the same comb.

Root-local parity, depth-local powers of two. The root window alone decides n mod 2 for n = 1 to 64: an even numeral has the doubling box at the root, an odd one the successor link. A depth-2 window decides n mod 4, and by induction divisibility by 2^j is decided at depth j. This is the first genuinely immediate divisibility in the project, and by Section 2.3 it is also close to the best a positional encoding can give.

Compactness lives in the DAG. Doubling duplicates its argument syntactically, so trees are linear while distinct subterms stay logarithmic: bin 1024 has 16,377 leaves as a tree but only 42 distinct subterms; bin 255 has 2,676 leaves and 40 distinct subterms. Sharing restores the logarithmic size that the tree loses.

The frozen product persists. Weak reduction sends B (bin 3)(bin 5) to S(K bin 3)(bin 5). The product box of [1] is representation-independent: it exists because weak reduction cannot enter the second argument of K, not because of anything the numerals do.

Doubling exists inside the system. Bracket abstraction with the eta shortcut turns the term for 'apply twice' into D = S(S(KS)(S(K(S(KS)K)))(SKK), and D(bin 3) weak-normalizes to exactly bin 6.

The orientation conflict. The arithmetic equality of [1] points the wrong way: its rewriting system R_arith flattens bin n to the unary comb (verified for 6, 7, and 12). The equality that created numbers in the predecessor paper destroys the structure this paper needs, so the systems below carry their own orientation.

5. Divisibility as local rewrite automata

For each divisor k there is a finite family of local rules over states (r, p) in Z_k by Z_k. A probe term

```
mark r p t  =  S(K(S(S(K r))(K p))) t
```

carries the accumulated residue r and the current bit weight p over an untouched binary numeral t. The wrapper is inert under weak reduction and introduces no R_arith redex. Three rule schemata walk the digit word from the least significant bit inward:

```
Mod0:  mark r p (dbl m)        ->  mark r (2p mod k) m
Mod1:  mark r p (suc (dbl m))  ->  mark ((r + p) mod k) (2p mod k) m
ModC:  mark r p one            ->  comb ((r + p) mod k)
```

Every applied instance preserves the affine invariant (r + p · value(t)) mod k, checked at each step by the harness.

Proposition 1. For every n at least 2 and every k at least 2, the probe started at mark 0 1(bin n) reaches the residue comb of n mod k in exactly bitlen(n) rule applications, each instance sound for the affine invariant.

Proof sketch. Each of Mod0 and Mod1 consumes one digit level and preserves the invariant by the two identities r + p(2m) = r + (2p)m and r + p(2m + 1) = (r + p) + (2p)m taken mod k; ModC consumes the terminator, whose contribution is p. The digit word of n has bitlen(n) levels including the terminator. The harness verifies the invariant at every step and the final residue against n mod k for all n = 2 to 40 and all k = 2 to n. ∎

Two consequences follow immediately. First, the residue vectors of [1] reproduce identically at a fraction of the cost: 12 steps against 35 for n = 6, 44 against 138 for n = 13, and 52 against 198 for n = 15. The dead-end anatomy of the predecessor paper transforms exactly as hoped: the residue is no longer a stranding shape reached after order n steps but the terminal state of an automaton, reached after logarithmically many. Second, primality certificates by probing k = 2 up to the integer square root: 97 is prime in 56 local rewrite steps, 1,009 in 300, 8,191 in 1,157, and the Fermat prime

65,537 in 4,335. Composites emit their smallest factor as the first zero residue: 91 gives 7, 143 gives 11, and 1,001 gives 7.

The picture is consistent with Section 2.3: each fixed k is a finite automaton, and primality needs the unbounded family of probes up to the square root. No single finite rule set can replace the family; only a different representation could.

6. F_arith: factoring as rewriting

6.1 The system and its three propositions

F_arith rewrites toward factored forms: product boxes S(K a)b over numeral leaves. One certified rule family splits a composite comb leaf of value v into S(K k)((v/k) as a comb) for any proper divisor k, and every applied instance is certified by normalizing the box forward under R_arith and comparing with the leaf it replaced, the discipline of [1] reused in reverse. Three canonicalization rules, Fassoc, Fsort, and Fswap, rotate boxes to the right and sort leaves ascending; each is pointwise valid, machine-checked. The canonical form of n is the ascending right-nested prime box tree.

Proposition 2 (primality is irreducibility). For n at least 2, the numeral of n admits no split move exactly when n is prime. A prime is a term on which the factoring system has nothing to say. Machine-checked for n = 2 to 200.

Proposition 3 (termination). F_arith terminates on numeral-generated forms: splits strictly decrease the multiset of leaf values, and the canonicalization rules decrease left depth and inversion count. This stands in pleasant contrast to the multiplying orientation R_arith, whose strong normalization remains open in [1].

Proposition 4 (confluence is unique factorization). Given the fundamental theorem of arithmetic, every maximal F_arith derivation from the numeral of n ends in the canonical ascending prime form, so the system is confluent on numerals; conversely, the joinability of split peaks instantiates unique factorization. The harness checks every first split of every composite n up to 100 (n = 60 alone has 10 ordered first splits) and fully randomized derivations over three seeds; all rejoin at the canonical form.

The probe-guided pipeline factors with certified steps: 30 into 2, 3, 5 in two splits (13 probe steps, 57 certification steps); 143 into 11, 13 in one split (80 and 166); 210 into 2, 3, 5, 7 in three splits (46 and 373); 1,001 into 7, 11, 13 in two splits (140 and 1,182); 1,024 into ten twos in nine splits (63 and 2,089). The prime 97 is irreducible at zero splits. The cost anatomy is exactly what the experiment was built to expose: discovery is cheap and certification is expensive, each unary certification of a split costing order v steps for a leaf of value v.

6.2 Binary leaves and rewrite division

The certificates then move from unary forward multiplication to division rewrites on binary leaves. Division by 2 is one root-local step: the doubling box at the root of an even leaf is itself the certificate, the parity locality of Section 4 cashed in. Division by an odd prime k runs least-significant-bit first with a bounded state r < k, a Hensel-style transition family reusing the probe shape of Section 5, one transition per binary digit, each checked against the all-natural-number invariant

```
k · Q  +  2^i · rest  =  v  +  2^i · r
```

A nonzero final state refuses non-divisors, and the quotient term is produced by the run itself, matching the canonical numeral exactly.

Proposition 5. For odd k dividing v, the division marker computes bin(v/k) from bin(v) in exactly bitlen(v) transitions, each preserving the invariant; for k = 2 one root-local step suffices.

The cost collapse across the pipeline: 30 drops from 57 unary certification steps to five division steps, 1,001 from 1,182 to 18, 1,024 from 2,089 to nine, and 4,095 from 6,046 to 39. For 65,535, which factors as 3, 5, 17, 257 and where unary certification is impractical, the full certified factorization costs 300 probe steps and 44 division steps.

With this, every ingredient of the factoring equality costs one step per binary digit except one: discovery still probes candidate divisors in ascending order, so the square-root wall of trial division remains. That is exactly where Observation 1 says the irreducible cost must live for a representation with cheap encoding. F_arith is, in this precise sense, optimal within its design point: the barrier has been squeezed into a single named location.

7. Reading K and S arithmetically

The philosophical route starts from what the two combinators do. K x y keeps its first argument and discards the second: separation, projection, hiding. S a b z feeds one input to two consumers: fusion, sharing, duplication.

The predecessor paper already runs on this reading without naming it. In the product box S(K m)k, the S at the root fuses the incoming argument into both branches, and the K shields m from it, so m receives k z rather than z. Composition is exactly 'share the input, hide it from the outer factor'; the frozen boxes are K-guarded potential energy, and R_arith spends it through the S-duplication in its multiplication rule, which is also why its termination measure fails. Section 4 adds the positive face of the same coin: S-sharing is why binary numerals are DAG-compact.

Two predictions follow. First, any design that K-guards a primality marker at the root must pay for maintaining it elsewhere, by Observation 1; there is no free flag. Second, and constructively: the S-as-fusion reading points at residue bundles. Represent n as an S-fused bundle of cyclic counters, one per small prime p, each counter a comb of length n mod p; K-projections read out a coordinate. Addition and multiplication act coordinatewise and locally; divisibility by every bundled prime is a root window on its coordinate; primality up to the smoothness bound is a conjunction of nonzero tests. The costs, hard comparison and blindness above the bound, are the ones the barrier forces anyway. Residue bundles are the K/S-native number system. The construction follows.

The construction, delivered. Fix a basis of primes, say 2, 3, 5, and 7 with modulus 210. A residue bundle is a right-nested chain of inert two-slot carriers, pair a b = S(S(K a))(K b), holding one cyclic residue comb per basis prime; the whole term is a weak and R_arith normal form of constant size for a fixed basis, 78 leaves for the bundle of 97. The philosophy of this section is visible as a reduction fact: applied to an argument X, the carrier reduces as pair a b X to a (X b), the root S fusing X into both branches while each K shields its stored residue. Machine-checked results (Appendix B):

- Divisibility by the i-th basis prime is a window read at depth i, the coordinate being the zero comb; verified for every n from 0 to 209 and every basis prime, and again for the six-prime basis with modulus 30,030.
- Primality is a conjunction of window reads on the validity window: for n from 2 to 120, n is prime exactly when it lies in the basis or every coordinate is nonzero; the six-prime basis extends the window to 288. Verified exhaustively.
- Arithmetic is coordinatewise and constant-cost: successor, addition, and multiplication are rule tables of at most p squared instances per coordinate, and every operation costs exactly one local rewrite per coordinate, four steps on the four-prime basis, independent of the operands. Verified against the modular semantics over all 3,600 operand pairs below 60.
- The encoder composes from Section 5: one divisibility probe per basis prime assembles the bundle of n from its binary numeral, and the bundle of 8,191 costs 78 local steps.

- The paid costs arrive on schedule: the bundle carries n modulo the basis product only, so the successor iterated 211 times from zero returns to the bundle of 1, the product of the bundles of 6 and 35 is the zero bundle, and comparison admits no bounded window. This is the ledger Observation 1 demands, and the design point that the table of Section 3.2 predicted is now occupied, at exactly the predicted price.

8. The enumerative search

Could a small representation nobody has thought of already reveal the primes in its own reduction? The question is decidable in the small, and Section 8 decides it there.

8.1 Method

Candidates are pairs of a base term and a constructor alphabet, and every candidate receives its meaning for free through iterator semantics: the value of a word is the n with word f x reducing weakly to f applied n times to x. This turns 'every possible representation' into a well-defined enumeration. Bases are closed terms that are iterators of 0 or 1. Constructors are enumerated two ways: since an action depends only on the constructor's weak normal form, all closed weak normal forms up to fifteen leaves, generated by the grammar N ::= K | S | K N | S N | S N N (278,707,802 terms), and additionally all raw terms up to nine leaves, which catches redex representatives whose normal forms are larger. Three ingredients make the scale feasible. Two pruning lemmas remove over ninety million candidates outright at the top sizes: every K-headed constructor is a constant map, S K x is the identity for every x, and S S x is never an iterator, because its evaluation stacks the iteration variable over a residual argument; the pruned classes were also scanned empirically at full scale and kept exactly zero candidates. A Krivine-style weak head machine with an argument stack replaces the root-rescanning reducer in the scan path, one unit of fuel per rule firing, reading the iterator value off the head spine; the original reducer remains in the rank stage as an independent cross-check on every keeper. A constructor is kept when its orbit from the base, or from the canonical numeral 2 for combination-only actions, yields distinct values within stated fuel and value caps, and keepers are deduplicated by orbit signature. With this, the entire frontier scans in about ten minutes of machine time, the largest single size in seven.

The screen is staged, smallest numbers first. An alphabet must cover the window 2 to 10 with 5 and 6 present, and passes to stage 2 (window 2 to 30) only if some cheap observable of its reduction behavior separates primes from composites exactly: minimal word length, representation count, normalization step count, normal form size, or a depth-2 root fingerprint. Two controls calibrate the screen: the unary alphabet must come out unpromising, and the product-box grammar of Section 6, run as a positive control, must separate primality exactly, which it does for n up to 30.

8.2 Results

The census. Across the whole frontier, 656 keepers reduce to 33 distinct orbit signatures; since a signature is an orbit prefix, budget-truncated duplicates of one map can appear twice, and at least 24 distinct arithmetic actions remain after identifying prefixes. The strata read like a small history of arithmetic. Five leaves: the successor S(S(KS)K) and squaring as S(S(SS))K, well below the textbook S B I at eight. Seven: the pronic map n(n + 1). Nine: doubling as S(K(S(S(K(SS))K)K)), where the hand-derived abstraction of Section 4 costs thirteen, plus n squared plus n plus 1, n cubed plus 1, and n squared times (n + 1). Eleven: n squared plus 1. Twelve, the richest stratum: the additive step n + 2 as S(K(SB))(SB), the honest 2n + 1 with orbit 1, 3, 7, 15 (the eta-safe form of the map whose naive S(SB) is weakly stuck), tripling with orbit 1, 3, 9, 27, the square of the successor (n + 1) squared, and the exponential 2 to the n with orbit 1, 2, 4, 16. Thirteen adds, among others, a map with orbit 1, 2, 16, consistent with 2 to the n squared. Fourteen, the newest stratum, contributes quadrupling with orbit 1, 4, 16, 64, the shifted exponential 2 to the (n + 1) with orbit 1, 4, 32, and the map 2 n squared with orbit 1, 2, 8, 128. Fifteen contributes, among others, the map n(2n + 1) with orbit 1, 3, 21. The lemma-

pruned classes kept exactly zero candidates at every size where they were also scanned empirically, including 5.25 million candidates at fourteen; at fifteen the lemmas excuse 71.7 million outright. The reduction cost of squaring remains a data point: producing the value 16 costs 253 weak steps, the value 256 costs 4,338, superlinear in the output.

A lesson about weak reduction. The natural candidate S(SB) for the map n to 2n + 1 is extensionally correct but weakly stuck: it applies the iteration variable f to a partial term, and the result is not an iterator. Honest actions may only compose, which drives the sizes of hand-derived abstractions up: the doubling of Section 4 costs thirteen leaves. The machine is a better combinator programmer than the abstraction algorithm, finding doubling at nine. Extensional intuition and weak-reduction reality diverge exactly at the eta boundary that [1] mapped.

The null result, thrice confirmed. Over the 40 smallest actions, 781 alphabets arise (singles and pairs over all, triples over the twelve smallest); 104 cover the window with 5 and 6 present. Across the campaign the staged screen has now promoted exactly two finite-window signals, a triple separating by representation counts at the eleven-leaf frontier and a pair of 2n + 1 with a doubling variant separating by normal form size at fifteen, and stage 2 executed both on the window 2 to 30; the first also dissolved retroactively under fuller evaluation budgets. Nothing else was ever promising, even with tripling, quadrupling, 2n + 1, and two exponentials in play. The pleasing closure stands: the alphabet of successor and doubling is the binary numeral system of Section 4, rediscovered blind, and its verdict is what the locality obstruction of Section 2.3 predicts. The grail, if it exists, is not below fifteen leaves. The next increments are the pool at sixteen leaves, about 0.95 billion normal forms, and a grammar extended by a binary composition node, which provably rediscovers the boxes of Section 6.

9. From verified runs to verified proofs

The machine-checking discipline of Appendix A is the weak sense of verification: a program computes each claim's instances and prints its findings. This section reports the upgrade to the strong sense for the paper's central schematic claims and, since both papers share one calculus, for the core theorems of [1]: statements become Idris2 types, proofs become total terms, and the compiler checks both once. The proof module Proofs.idr, grown to 3,337 lines since this paper's first pass and shared by every paper of the series, is mapped in Appendix C; it is declared %default total and contains no postulates and no holes, so the command idris2 —check Proofs.idr is the entire verification. Where Sections 4 to 8 validated instances, everything below is quantified over all terms and all numbers.

The bedrock is a change of authority. Reduction facts are no longer the evaluator's report; they are inhabitants of an inductive relation, one constructor per rule and two for closure under application contexts:

```
data Step : Term -> Term -> Type where
  RedK : Step (App (App CK a) b) a
  RedS : Step (App (App (App CS a) b) c) (App (App a c) (App b c))
  CtxL : Step l l' -> Step (App l r) (App l' r)
  CtxR : Step r r' -> Step (App l r) (App l r')

data Steps : Term -> Term -> Type where
  Done : Steps t t
  Then : Step s t -> Steps t u -> Steps s u
```

Three pieces of infrastructure carry the theorems. Congruence lifts and transitivity assemble long derivations from short ones. A substitution lemma, immediate because variables take no step, makes every chain proved at generic parameters a genuinely universal statement. And a certified

evaluator returns each normal form together with its derivation, wnfCert : Nat -> (t : Term) -> (t' ** Steps t t'), which demotes the fueled evaluator of Appendix B from arbiter to untrusted oracle whose every answer carries its own proof.

For the companion system of [1], the module proves the adequacy of the numerals and the pointwise validity of all four arithmetic rules, universally in their parameters, and Theorem A end to end:

```
numAdequate : (n : Nat) -> (x : Term) ->
  Steps (App (App (num n) (Var "f")) x) (fpow n x)

r1Valid : (u, v : Term) ->
  Joinable (App (App (r1L u v) (Var "f")) (Var "z"))
           (App (App (r1R u v) (Var "f")) (Var "z"))

leavesGraft : (s, t : LTree) ->
  leaves (graft s t) = leaves s * leaves t
```

Theorem A is formalized constructively as the equivalence between composite sizes and decomposable trees, both directions witness-producing, with the prime formulation recovered as corollaries; no appeal to decidability is needed.

For the search of Section 8, the three structural lemmas quoted there are now theorems. That a K-headed candidate is constant is a single constructor. That S K x is the identity is a two-step chain, quantified over the payload and the argument. And the sharpest one, that S S x is never an iterator, is formalized in three precise pieces: ssReach shows that the canonical reduct of S S x applied to any iterator seed has the stuck root shape, the iteration variable applied to two arguments; fh2Steps shows that this shape is preserved by every further reduction step, at any position; and the corollary closes the door:

```
ssStuckForever : {n : Nat} -> FHead2 r ->
  Steps r (fpow n (Var "x")) -> Void
```

One honest gap is recorded rather than papered over: the statement quantified over all reduction strategies from the original term would additionally need Church-Rosser for weak reduction, and that classical theorem is scheduled, not assumed.

For F_arith, the module proves the value invariant, the termination engine, and Proposition 2 at the leaf where numerals live:

```
fStepVal       : FStep t u -> fval t = fval u
splitDecreases : SplitStep t u -> LT (mu u) (mu t)
```

together with primeIrreducible and irreduciblePrime, the constructive equivalence between a leaf admitting a split and its value being composite. One design choice made the invariant partly definitional: the Split rule is indexed by the factored value written as successors, S (S k) * S (S q), so both factors are at least 2 by construction and value preservation at splits is Refl. The fusion fact of Section 7, pair a b X reducing to a (X b), is a four-step certified chain.

Tier two, delivered. Since version 0.8 the second tier sits in the same module, which has since grown to 3,337 lines. Proposition 1 is a theorem in divmod form. The probe runs over the value-indexed binary view BinV : Nat -> Type, least significant bit outermost; both registers stay strictly below k, so each update needs at most one restoring subtraction, and the affine invariant of Section 4 threads through the run as a compiled equation:

```
probeCert : (k : Nat) -> LTE 2 k -> {n : Nat} -> BinV n ->
  (r ** q ** (LT r k, n = r + q * k))
```

The returned register is the remainder of n by k, quotient attached as certificate, and the recursion consumes exactly one view constructor per step, so the cost is bitLen n by construction. No modulus function is defined anywhere; the divmod specification replaced the planned modular arithmetic lemma kit, which shrank to eight Nat identities. The well-founded packaging of Proposition 3 is delivered as splitTerminates : (t : FForm) -> Accessible SplitRev t, every sequence of splits terminates, by strong induction on the leaf measure. The fragment form of Theorem B of [1] is delivered through the value invariant exactly as planned, and without confluence: arithmetic forms are an inductive family AF : Nat -> Term -> Type indexed by their value, R steps preserve the index (stepRAF), the index at a numeral is unique (afUnique), and with the schematic forward derivation boxReduces this closes both directions of the reachability characterization. Finally, the finite-exhaustion bridge allBelowSound lifts one verified run over an enumerated domain to the bounded universal statement, the schema under which the census claims can be restated as theorems relative to a checked run.

What changes, and what does not. The finite claims of the paper, the census and the null result of Section 8 and the exhaustive windows of Section 7, remain exactly what they were, checks over stated bounds, and no universality is claimed for them. The schematic claims above, by contrast, are now theorems checked once at compile time and never re-run. What remains from the second tier is Proposition 5′ s division certificates, which follow the probeCert pattern, and the bundle correctness of Section 7 for all n rather than the swept ranges. The third tier holds the fundamental theorem of arithmetic as a standalone module, which would discharge the hypothesis of Proposition 4, and Church-Rosser. The toolchain is Idris2 0.8.0 [11] with the base library only.

10. Relation to other work

The predecessor paper [1] surveys the lineage this project extends: numeral systems in the lambda calculus and combinatory logic, arithmetic as term rewriting, and arithmetics defined directly on trees, with Bimbó [2] and Baader and Nipkow [3] as the standing references for combinatory logic and rewriting. This paper adds three threads.

Automata and primes. The locality obstruction of Section 2.3 rests on three classical results: Minsky and Papert proved the primes are not regular in any base [4], Hartmanis and Shank extended this to context-free languages and to every infinite subset of the primes [5], and Shallit gave the quantitative version, a polynomial state lower bound for automata correct up to N [6]. Section 5 sits exactly on the boundary these results draw: divisibility by each fixed k is a finite automaton and becomes a finite family of local rewrite rules, while primality needs the unbounded family, which no representation change within positional encodings can compress.

Testing versus extraction. The asymmetry of Section 2.2 is the AKS theorem [7]. The barrier of Observation 1 is folklore in spirit; we have not found it stated in this representation-theoretic form, and it does useful work here, organizing the design space of Section 3.2 and localizing the residual cost of Section 6.2.

Exact division and residue arithmetic. The least-significant-bit-first division of Section 6.2 is the rewriting form of exact division by odd constants via 2-adic inversion, the mechanism behind Montgomery's modular multiplication without trial division [8]. Residue number systems, the target of the construction predicted in Section 7, are classical [9]; what would be new is their realization as the native number system of separation and fusion inside {K, S}.

11. Conclusion

The paper set out to make primality and factorization immediate in a combinator representation, and it returns with the boundary of the possible mapped and the possible built. Three external facts fence the field: the conversion barrier, the testing-extraction asymmetry, and the locality obstruction for positional encodings. Inside the fence, the machinery now exists. Binary numerals make divisibility by powers of two literally visible at the root. Divisibility by any fixed k is a three-rule local automaton whose certificates cost one step per binary digit. The factoring system F_arith makes primality irreducibility and unique factorization confluence, terminates provably, and, with rewrite division on binary leaves, certifies every split in one step per digit. A five-digit number is factored, with every step machine-verified, in a few hundred local rewrites. The single cost that remains, the ascending search for a divisor, is the one Observation 1 proves must remain somewhere.

What the paper does not deliver is the grail itself: a representation whose canonical forms wear primality as a bounded pattern. Section 2.3 proves no positional encoding can, and Section 8 verifies that nothing at the searched frontier does, all weak normal forms to fifteen leaves and all raw terms to nine, while calibrating the search that continues upward. The residue bundles are delivered in Section 7. What remains queued is the pool at sixteen leaves on the search side, and, on the construction side, the growing-basis version of the bundles and the remainder of the formalization program of Section 9.

Honest limitations, recorded as in [1]. Proposition 2, the strict decrease at the heart of Proposition 3, and the value invariant at the base of Proposition 4 are now compiler-checked theorems, quantified over all inputs (Section 9, Appendix C). Proposition 1 and the well-founded packaging of Proposition 3 joined them in version 0.8, the former in divmod form over the binary view, the latter as constructive accessibility. Proposition 5 and the discharge of Proposition 4' s arithmetic hypothesis remain at the level of value semantics and machine-instantiated initial segments; they are the scheduled remainder of the same program. The census of Section 8 is exhaustive relative to generous but finite fuel and value caps, and the stated bound, no further healthy action below the caps, is what the machine established. The probe and division rule families are sound for their invariants and deliberately not extensional identities; they are observers, not members, of the number equality.

The thesis of the project, that the equality does the work, survives its second test. In the predecessor paper the right equality created the numbers; here the right orientation of an equality turns the fundamental theorem of arithmetic into confluence, and moves every cost of factoring except one into places where a machine can count it.

Appendix A: Methods

All experiments run in a single Idris2 module, Secret2.idr, reproduced verbatim in Appendix B, compiled with Idris2 0.8.0 on the Chez Scheme backend and executed with idris2 Secret2.idr -x main. The module contains the term language, weak reduction and R_arith from [1], the binary numerals and their probes, the F_arith system with certified splits and rewrite division, and the staged enumerative search. Every figure quoted in the paper is printed by the run; the complete output accompanies the paper. The frontier search of Section 8 runs through two further entry points of the same binary: e4scan performs chunked and structurally partitioned candidate scanning with keepers persisted to disk, and e4rank deduplicates by orbit signature, re-verifies every keeper under the original reducer, and applies the staged screen. One engineering note earned its place in the record: the standard library concatenation is left-folded, which turns list enumeration quadratic near outer lists of 10^5 elements; a right-folded concatenation restored linearity and turned a candidate class that had hung for minutes into a one-second scan. The machine-checking discipline follows [1]: every split is certified against the forward system or a step invariant, every probe and

division transition is validated against its affine invariant at the moment it is applied, and negative claims (the census, the null separation) are exhaustive relative to stated caps that the source records as constants. The proof module of Section 9, Proofs.idr, is the series' shared artifact, mapped in Appendix C and checked separately with idris2 —check Proofs.idr; it is total by declaration, imports only the base library, and contains no postulates or holes, so a clean check is the complete verification.

Appendix B: Idris2 source

The consolidated experiment module, verbatim.

```
||| Project secret2, experiment E1 : binary numerals from doubling and successor.
||| Calibration for the search for a combinator representation with
||| transparent divisibility, primality, and factorization.
||| Run with:  idris2 Secret2.idr -x main
module Secret2

import Data.List
import Data.Maybe
import Data.String
import Data.SortedMap
import Data.SortedSet
import System
import System.File

%default covering

--------------------------------------------------------------------------
-- helpers
--------------------------------------------------------------------------

upTo : Nat -> Nat -> List Nat
upTo lo hi = map (lo +) (below (minus (S hi) lo))
  where
    below : Nat -> List Nat
    below Z     = []
    below (S k) = below k ++ [k]

getAt' : Nat -> List a -> Maybe a
getAt' _     []        = Nothing
getAt' Z     (x :: _)  = Just x
getAt' (S i) (_ :: xs) = getAt' i xs

cmap : (a -> List b) -> List a -> List b
cmap f []        = []
cmap f (x :: xs) = f x ++ cmap f xs

boolMark : Bool -> String
boolMark True  = "OK"
boolMark False = "FAIL"

divN : Nat -> Nat -> Nat
divN n k = fromInteger (div (cast n) (cast k))

modN : Nat -> Nat -> Nat
modN n k = fromInteger (mod (cast n) (cast k))
```

```
-----------------------------------------------------------------------
-- combinatory terms (as in Secret1)
-----------------------------------------------------------------------

data Term = CS | CK | Var String | App Term Term

Eq Term where
  CS      == CS      = True
  CK      == CK      = True
  Var a   == Var b   = a == b
  App a b == App c d = a == c && b == d
  _       == _       = False

Ord Term where
  compare CS CS           = EQ
  compare CS _            = LT
  compare _  CS           = GT
  compare CK CK           = EQ
  compare CK _            = LT
  compare _  CK           = GT
  compare (Var a) (Var b) = compare a b
  compare (Var _) _       = LT
  compare _ (Var _)       = GT
  compare (App a b) (App c d) =
    case compare a c of
      EQ => compare b d
      o  => o

Show Term where
  show CS      = "S"
  show CK      = "K"
  show (Var v) = v
  show (App l r) =
    show l ++ (case r of
                 App _ _ => "(" ++ show r ++ ")"
                 _       => show r)

iC, bC, ki, sb, skb : Term
iC  = App (App CS CK) CK
bC  = App (App CS (App CK CS)) CK
ki  = App CK iC
sb  = App CS bC
skb = App CS (App CK bC)

unary : Nat -> Term
unary Z     = ki
unary (S n) = App sb (unary n)

prodBox : Term -> Term -> Term
prodBox m k = App (App CS (App CK m)) k

multB : Term -> Term -> Term
multB m n = App (App bC m) n
```

```
tsize : Term -> Nat
tsize (App l r) = tsize l + tsize r
tsize _         = 1

-----------------------------------------------------------------------
-- weak reduction and R_arith (as in Secret1)
-----------------------------------------------------------------------

matchW : Term -> Maybe Term
matchW (App (App CK a) _)         = Just a
matchW (App (App (App CS a) b) c) = Just (App (App a c) (App b c))
matchW _                          = Nothing

stepW : Term -> Maybe Term
stepW t =
  case matchW t of
    Just r  => Just r
    Nothing =>
      case t of
        App l r =>
          case stepW l of
            Just l' => Just (App l' r)
            Nothing => map (App l) (stepW r)
        _ => Nothing

wnf : Term -> Term
wnf = go 400000
  where
    go : Nat -> Term -> Term
    go Z     t = t
    go (S f) t = case stepW t of
                   Nothing => t
                   Just t' => go f t'

matchRoot : Term -> Maybe (String, Term)
matchRoot (App (App CK a) _) = Just ("wK", a)
matchRoot (App (App (App CS a) b) c) =
  Just ("wS", App (App a c) (App b c))
matchRoot (App (App CS (App CK y)) v) =
  if y == ki
    then Just ("R2", ki)
    else case y of
           App h u => if h == sb
                        then Just ("R1",
                               App (App CS (App skb v))
                                   (App (App CS (App CK u)) v))
                        else Nothing
           _ => Nothing
matchRoot (App (App CS (App h z)) v) =
  if h == skb
    then if z == ki
           then Just ("R4", v)
           else case z of
                  App h2 u => if h2 == sb
                                then Just ("R3",
```

```
                                      App sb (App (App CS (App skb u)) v))
                                else Nothing
                  _ => Nothing
    else Nothing
matchRoot _ = Nothing

stepR : Term -> Maybe Term
stepR t =
  case matchRoot t of
    Just (_, r) => Just r
    Nothing     =>
      case t of
        App l r =>
          case stepR l of
            Just l' => Just (App l' r)
            Nothing => map (App l) (stepR r)
        _ => Nothing

nfR : Term -> Term
nfR = go 400000
  where
    go : Nat -> Term -> Term
    go Z     t = t
    go (S f) t = case stepR t of
                   Nothing => t
                   Just t' => go f t'

-----------------------------------------------------------------------
-- bracket abstraction, with the eta shortcut
-----------------------------------------------------------------------

occurs : String -> Term -> Bool
occurs x (Var y)   = x == y
occurs x (App a b) = occurs x a || occurs x b
occurs _ _         = False

abstr : String -> Term -> Term
abstr x t =
  if not (occurs x t)
    then App CK t
    else case t of
           Var _   => iC
           App a b =>
             if b == Var x && not (occurs x a)
               then a
               else App (App CS (abstr x a)) (abstr x b)
           _ => App CK t

-----------------------------------------------------------------------
-- binary numerals : doubling and successor
-----------------------------------------------------------------------

dblT : Term -> Term                      -- S(S(KB)m)m, the self-sum box
dblT m = App (App CS (App skb m)) m
```

```
sucT : Term -> Term
sucT m = App sb m

bin : Nat -> Term                        -- defined for n >= 1
bin n =
  if n <= 1 then unary 1
  else if modN n 2 == 0
    then dblT (bin (divN n 2))
    else sucT (dblT (bin (divN n 2)))

-- the doubling combinator, derived by abstraction of \n.\f. B (n f) (n f)
dComb : Term
dComb = abstr "n" (abstr "f"
          (App (App bC (App (Var "n") (Var "f"))) (App (Var "n") (Var "f"))))

-----------------------------------------------------------------
-- structural probes
-----------------------------------------------------------------

evenChild : Term -> Maybe Term           -- Just m iff root is the doubling box
evenChild (App (App CS (App h m)) y) =
  if h == skb && m == y then Just m else Nothing
evenChild _ = Nothing

parityLocal : Nat -> Bool                -- root window decides n mod 2
parityLocal n = isJust (evenChild (bin n)) == (modN n 2 == 0)

mod4Local : Nat -> Bool                  -- depth-2 window decides n mod 4, n even
mod4Local n =
  case evenChild (bin n) of
    Just m  => isJust (evenChild m) == (modN n 4 == 0)
    Nothing => False

countDistinct : Term -> Nat              -- DAG size: distinct subterms
countDistinct t = snd (go t (SortedSet.empty, Z))
  where
    go : Term -> (SortedSet Term, Nat) -> (SortedSet Term, Nat)
    go u (s, n) =
      if SortedSet.contains u s
        then (s, n)
        else let s' = SortedSet.insert u s in
             case u of
               App a b => go b (go a (s', S n))
               _       => (s', S n)

adequate : Nat -> Bool                   -- bin n and unary n agree pointwise
adequate n =
  wnf (App (App (bin n) (Var "f")) (Var "x"))
    == wnf (App (App (unary n) (Var "f")) (Var "x"))

-----------------------------------------------------------------
-- E2 : divisibility automata as local rewrite rules
-----------------------------------------------------------------

combLen : Term -> Maybe Nat
```

```
combLen t =
  if t == ki
    then Just Z
    else case t of
           App h u => if h == sb then map S (combLen u) else Nothing
           _       => Nothing

binVal : Term -> Maybe Nat               -- decode a canonical binary numeral
binVal t =
  if t == unary 1
    then Just 1
    else case evenChild t of
      Just m  => map (\v => 2 * v) (binVal m)
      Nothing =>
        case t of
          App h x =>
            if h == sb
              then case evenChild x of
                     Just m  => map (\v => S (2 * v)) (binVal m)
                     Nothing => Nothing
              else Nothing
          _ => Nothing

bitLen : Nat -> Nat
bitLen n = if n <= 1 then 1 else S (bitLen (divN n 2))

isqrt : Nat -> Nat
isqrt n = go Z
  where
    go : Nat -> Nat
    go s = if (S s) * (S s) > n then s else go (S s)

-- the probe: S(K(S(K r)p))t, a frozen box carrying state (r, p) over t
mkMark : Term -> Term -> Term -> Term
mkMark r p t =
  App (App CS (App CK (App (App CS (App CS (App CK r))) (App CK p)))) t

unmark : Term -> Maybe (Term, Term, Term)
unmark (App (App CS (App CK (App (App CS (App CS (App CK r))) (App CK p)))) t) =
  Just (r, p, t)
unmark _ = Nothing

-- one rule application from the finite family for divisor k:
--   Mod0: mark r p (dbl m)       -> mark r (2p mod k) m
--   Mod1: mark r p (suc (dbl m)) -> mark ((r+p) mod k) (2p mod k) m
--   ModC: mark r p one           -> comb ((r+p) mod k)
stepMod : Nat -> Term -> Maybe (String, Term)
stepMod k mt = do
  (rT, pT, t) <- unmark mt
  r <- combLen rT
  p <- combLen pT
  if t == unary 1
    then Just ("ModC", unary (modN (r + p) k))
    else case evenChild t of
      Just m  => Just ("Mod0", mkMark (unary r) (unary (modN (2 * p) k)) m)
```

```
      Nothing =>
        case t of
          App h x =>
            if h == sb
              then case evenChild x of
                     Just m => Just ("Mod1",
                                 mkMark (unary (modN (r + p) k))
                                        (unary (modN (2 * p) k)) m)
                     Nothing => Nothing
              else Nothing
          _ => Nothing

-- the invariant (r + p * value(t)) mod k; residue combs read directly
markVal : Nat -> Term -> Maybe Nat
markVal k mt =
  case unmark mt of
    Just (rT, pT, t) => do
      r <- combLen rT
      p <- combLen pT
      v <- binVal t
      pure (modN (r + p * v) k)
    Nothing => map (\r => modN r k) (combLen mt)

probeState : Nat -> Term -> String
probeState k mt =
  case unmark mt of
    Just (rT, pT, t) =>
      "r=" ++ show (fromMaybe 999 (combLen rT))
      ++ ",p=" ++ show (fromMaybe 999 (combLen pT))
      ++ " |" ++ show (fromMaybe 0 (binVal t))
    Nothing => "residue " ++ show (fromMaybe 999 (combLen mt))

record Probe where
  constructor MkProbe
  pResidue : Nat
  pSteps   : Nat
  pInvOK   : Bool
  pTrace   : List String

runProbe : Nat -> Nat -> Probe
runProbe k n =
  let start = mkMark (unary 0) (unary 1) (bin n)
  in go start Z (markVal k start) [probeState k start] (bitLen n + 4)
  where
    go : Term -> Nat -> Maybe Nat -> List String -> Nat -> Probe
    go cur st inv tr Z = MkProbe 999 st False (reverse tr)
    go cur st inv tr (S fuel) =
      case stepMod k cur of
        Nothing =>
          MkProbe (fromMaybe 999 (combLen cur)) st
                  (isJust inv && markVal k cur == inv) (reverse tr)
        Just (nm, nxt) =>
          if markVal k nxt == inv
            then go nxt (S st) inv
                    ((nm ++ " " ++ probeState k nxt) :: tr) fuel
```

```
            else MkProbe 998 (S st) False (reverse tr)

residueVecE2 : Nat -> (List Nat, Nat, Bool)
residueVecE2 n =
  let rs = map (\k => runProbe k n) (upTo 2 (minus n 1))
  in ( map (\r => r.pResidue) rs
     , sum (map (\r => r.pSteps) rs)
     , all (\r => r.pInvOK) rs )

certify : Nat -> (Bool, Maybe Nat, Nat, Nat)   -- (prime, factor, probes, steps)
certify n =
  let ks  = upTo 2 (isqrt n)
      rs  = map (\k => (k, runProbe k n)) ks
      tot = sum (map (\kr => (snd kr).pSteps) rs)
      inv = all (\kr => (snd kr).pInvOK) rs
      zs  = filter (\kr => (snd kr).pResidue == 0) rs
  in case zs of
       []             => (inv, Nothing, length ks, tot)
       ((k, _) :: _)  => (False, Just k, length ks, tot)

-----------------------------------------------------------------------
-- E3 : F_arith, the factoring orientation
-----------------------------------------------------------------------

data Dir = DL | DR

getAt : Term -> List Dir -> Maybe Term
getAt t          []        = Just t
getAt (App l _) (DL :: p)  = getAt l p
getAt (App _ r) (DR :: p)  = getAt r p
getAt _          _         = Nothing

putAt : Term -> List Dir -> Term -> Maybe Term
putAt _         []        new = Just new
putAt (App l r) (DL :: p) new = map (\l' => App l' r) (putAt l p new)
putAt (App l r) (DR :: p) new = map (App l)           (putAt r p new)
putAt _         _         _   = Nothing

unbox : Term -> Maybe (Term, Term)       -- product box S(K a)b
unbox (App (App CS (App CK a)) b) = Just (a, b)
unbox _                           = Nothing

||| positions of the leaves of the product-box structure
fpositions : Term -> List (List Dir)
fpositions t =
  case unbox t of
    Just (a, b) => map ([DL, DR, DR] ++) (fpositions a)
                ++ map ([DR] ++) (fpositions b)
    Nothing     => [[]]

fval : Term -> Maybe Nat                 -- product of leaf values
fval t =
  case unbox t of
    Just (a, b) => do va <- fval a; vb <- fval b; pure (va * vb)
    Nothing     => combLen t
```

```
-- meta reference: least prime factor, canonical factored form
lpf : Nat -> Nat
lpf n = go 2
  where
    go : Nat -> Nat
    go k = if k * k > n then n
           else if modN n k == 0 then k else go (S k)

factList : Nat -> List Nat
factList n = if n <= 1 then [] else lpf n :: factList (divN n (lpf n))

canonF : Nat -> Term
canonF n = build (factList n)
  where
    build : List Nat -> Term
    build []        = unary 1
    build [p]       = unary p
    build (p :: ps) = prodBox (unary p) (build ps)

showFact : Nat -> String
showFact n = concat (intersperse " * " (map show (factList n)))

nfRc : Term -> (Term, Nat)               -- R_arith normal form with step count
nfRc = go 400000 Z
  where
    go : Nat -> Nat -> Term -> (Term, Nat)
    go Z     c t = (t, c)
    go (S f) c t = case stepR t of
                     Nothing => (t, c)
                     Just t' => go f (S c) t'

||| all F-split redexes: composite comb leaves with each proper divisor
splitMoves : Term -> List (List Dir, Nat)
splitMoves t = cmap moves (fpositions t)
  where
    moves : List Dir -> List (List Dir, Nat)
    moves p =
      case getAt t p >>= combLen of
        Just v  => if v >= 4
                     then map (\k => (p, k))
                              (filter (\k => modN v k == 0) (upTo 2 (minus v 1)))
                     else []
        Nothing => []

||| certified split: leaf v -> S(K k)(v/k), verified by forward R_arith
doSplit : Term -> List Dir -> Nat -> Maybe (Term, Nat)
doSplit t p k = do
  leaf <- getAt t p
  v    <- combLen leaf
  if k >= 2 && modN v k == 0 && divN v k >= 2
    then let new = prodBox (unary k) (unary (divN v k))
             (chk, st) = nfRc new
         in if chk == leaf
              then map (\t' => (t', st)) (putAt t p new)
```

```
              else Nothing
    else Nothing

-- canonicalization: associate to the right, sort leaves ascending
saRoot : Term -> Maybe (String, Term)
saRoot t =
  case unbox t of
    Nothing => Nothing
    Just (a, b) =>
      case unbox a of
        Just (x, y) => Just ("Fassoc", prodBox x (prodBox y b))
        Nothing =>
          case combLen a of
            Nothing => Nothing
            Just va =>
              case unbox b of
                Just (bl, br) =>
                  case combLen bl of
                    Just vb => if vb < va
                                 then Just ("Fsort", prodBox bl (prodBox a br))
                                 else Nothing
                    Nothing => Nothing
                Nothing =>
                  case combLen b of
                    Just vb => if vb < va
                                 then Just ("Fswap", prodBox b a)
                                 else Nothing
                    Nothing => Nothing

stepSA : Term -> Maybe (String, Term)
stepSA t =
  case saRoot t of
    Just r  => Just r
    Nothing =>
      case unbox t of
        Just (a, b) =>
          case stepSA a of
            Just (nm, a') => Just (nm, prodBox a' b)
            Nothing       => map (\nb => (fst nb, prodBox a (snd nb))) (stepSA b)
        Nothing => Nothing

saFix : Term -> (Term, Nat)
saFix = go 4000 Z
  where
    go : Nat -> Nat -> Term -> (Term, Nat)
    go Z     c t = (t, c)
    go (S f) c t = case stepSA t of
                     Nothing      => (t, c)
                     Just (_, t') => go f (S c) t'

pointEq : Term -> Term -> Bool
pointEq a b =
  wnf (App (App a (Var "f")) (Var "z")) == wnf (App (App b (Var "f")) (Var "z"))

-- probe-guided divisor discovery (E2 reused)
```

```
discoverLpf : Nat -> (Nat, Nat, Bool)    -- (least factor or v, probe steps,
invariants)
discoverLpf v = go 2 Z True
  where
    go : Nat -> Nat -> Bool -> (Nat, Nat, Bool)
    go k acc inv =
      if k * k > v then (v, acc, inv)
      else let pr = runProbe k v
               acc' = acc + pr.pSteps
               inv' = inv && pr.pInvOK
           in if pr.pResidue == 0 then (k, acc', inv') else go (S k) acc' inv'

record FactRun where
  constructor MkFact
  frSplits : Nat
  frProbe  : Nat
  frCert   : Nat
  frSort   : Nat
  frOK     : Bool

firstComposite : Term -> Maybe (List Dir, Nat)
firstComposite t = go (fpositions t)
  where
    go : List (List Dir) -> Maybe (List Dir, Nat)
    go []        = Nothing
    go (p :: ps) = case getAt t p >>= combLen of
                     Just v  => if v >= 4 && lpf v /= v then Just (p, v) else go ps
                     Nothing => go ps

factorPipeline : Nat -> FactRun
factorPipeline n = loop (unary n) Z Z Z True (S (n + 4))
  where
    loop : Term -> Nat -> Nat -> Nat -> Bool -> Nat -> FactRun
    loop t s ps cs inv Z = MkFact s ps cs Z False
    loop t s ps cs inv (S fuel) =
      case firstComposite t of
        Nothing =>
          let (t2, ss) = saFix t
          in MkFact s ps cs ss (inv && t2 == canonF n && fval t2 == Just n)
        Just (p, v) =>
          let (k, pst, iv) = discoverLpf v
          in case doSplit t p k of
               Just (t', cst) => loop t' (S s) (ps + pst) (cs + cst) (inv && iv)
fuel
               Nothing        => MkFact s ps cs Z False

-- confluence harness
finishDet : Nat -> Term -> Term          -- complete with meta lpf, then
canonicalize
finishDet n t = go t (S (n + 4))
  where
    go : Term -> Nat -> Term
    go u Z = u
    go u (S fuel) =
      case firstComposite u of
```

```
        Nothing     => fst (saFix u)
        Just (p, v) => case doSplit u p (lpf v) of
                         Just (u', _) => go u' fuel
                         Nothing      => u

firstSplitsConfluent : Nat -> Bool
firstSplitsConfluent n =
  all (\pk => case doSplit (unary n) (fst pk) (snd pk) of
                Just (t1, _) => finishDet n t1 == canonF n
                Nothing      => False)
      (splitMoves (unary n))

lcg : Nat -> Nat
lcg s = modN (1103515245 * s + 12345) 2147483648

pick : Nat -> List a -> Maybe a
pick _ []        = Nothing
pick Z (x :: _)  = Just x
pick (S i) (_ :: xs) = pick i xs

randRun : Nat -> Nat -> Bool             -- random split order reaches canonF n
randRun seed n = go (unary n) seed (S (n + 4))
  where
    go : Term -> Nat -> Nat -> Bool
    go t s Z = False
    go t s (S fuel) =
      case splitMoves t of
        [] => fst (saFix t) == canonF n
        ms =>
          let s' = lcg s in
          case pick (modN s' (length ms)) ms of
            Just (p, k) => case doSplit t p k of
                             Just (t', _) => go t' s' fuel
                             Nothing      => False
            Nothing => False

-----------------------------------------------------------------------
-- E3.2 : binary leaves and rewrite division
-----------------------------------------------------------------------

pow2 : Nat -> Nat
pow2 Z     = 1
pow2 (S k) = 2 * pow2 k

peelBit : Term -> Maybe (Nat, Term)      -- root bit and payload; one => (1, ki)
peelBit t =
  if t == unary 1
    then Just (1, ki)
    else case evenChild t of
      Just m  => Just (0, m)
      Nothing =>
        case t of
          App h x => if h == sb then map (\m => (1, m)) (evenChild x) else Nothing
          _       => Nothing
```

```
record DivRes where
  constructor MkDiv
  dQ     : Term
  dRem   : Nat
  dSteps : Nat
  dInvOK : Bool

||| LSB-first exact division by odd k, as marker transitions with a
||| bounded state r < k; invariant k*Q + 2^i * rest == v + 2^i * r.
runDiv : Nat -> Nat -> Term -> DivRes
runDiv k v leaf = go (mkMark (unary 0) ki leaf) Z Z True (bitLen v + 4)
  where
    val0 : Term -> Nat
    val0 t = if t == ki then 0 else fromMaybe 0 (binVal t)
    invHolds : Nat -> Term -> Bool
    invHolds i mt =
      case unmark mt of
        Just (rT, qT, t) =>
          let r = fromMaybe 999 (combLen rT)
              w = pow2 i
          in k * val0 qT + w * val0 t == v + w * r && r < k
        Nothing => False
    go : Term -> Nat -> Nat -> Bool -> Nat -> DivRes
    go mt i st inv Z = MkDiv ki 999 st False
    go mt i st inv (S fuel) =
      case unmark mt of
        Nothing => MkDiv ki 998 st False
        Just (rT, qT, t) =>
          if t == ki
            then MkDiv (if qT == ki then ki else qT)
                       (fromMaybe 997 (combLen rT)) st
                       (inv && invHolds i mt)
            else case (combLen rT, peelBit t) of
              (Just r, Just (b, rest)) =>
                let q    = modN (r + b) 2
                    need = r + k * q
                    r'   = divN (minus need b) 2
                    qv   = val0 qT + q * pow2 i
                    qT'  = if qv == 0 then ki else bin qv
                    mt'  = mkMark (unary r') qT' rest
                    ok   = inv && invHolds i mt
                           && need >= b && modN (minus need b) 2 == 0
                in go mt' (S i) (S st) ok fuel
              _ => MkDiv ki 996 st False

||| certified binary split of a leaf of value v by a prime k:
||| k = 2 peels the doubling box in one root-local step,
||| odd k runs the division marker; returns (new leaf, steps)
splitLeafB : Nat -> Nat -> Term -> Maybe (Term, Nat)
splitLeafB k v leaf =
  if k == 2
    then case evenChild leaf of
           Just m => if binVal m == Just (divN v 2)
                       then Just (prodBox (bin 2) m, 1)
                       else Nothing
```

```
           Nothing => Nothing
    else
      let d = runDiv k v leaf
      in if d.dRem == 0 && d.dInvOK && d.dQ == bin (divN v k)
           then Just (prodBox (bin k) d.dQ, d.dSteps)
           else Nothing

canonB : Nat -> Term
canonB n = build (factList n)
  where
    build : List Nat -> Term
    build []        = bin 1
    build [p]       = bin p
    build (p :: ps) = prodBox (bin p) (build ps)

firstCompositeB : Term -> Maybe (List Dir, Nat)
firstCompositeB t = go (fpositions t)
  where
    go : List (List Dir) -> Maybe (List Dir, Nat)
    go []        = Nothing
    go (p :: ps) = case getAt t p >>= binVal of
                     Just v  => if v >= 4 && lpf v /= v then Just (p, v) else go ps
                     Nothing => go ps

record FactB where
  constructor MkFactB
  fbSplits : Nat
  fbProbe  : Nat
  fbDiv    : Nat
  fbOK     : Bool

factorPipelineB : Nat -> FactB
factorPipelineB n = loop (bin n) Z Z Z True (S (bitLen n * 4 + 8))
  where
    loop : Term -> Nat -> Nat -> Nat -> Bool -> Nat -> FactB
    loop t s ps ds inv Z = MkFactB s ps ds False
    loop t s ps ds inv (S fuel) =
      case firstCompositeB t of
        Nothing => MkFactB s ps ds (inv && t == canonB n)
        Just (p, v) =>
          let (k, pst, iv) = discoverLpf v
          in case getAt t p of
               Nothing => MkFactB s ps ds False
               Just leaf =>
                 case splitLeafB k v leaf of
                   Nothing => MkFactB s ps ds False
                   Just (new, dst) =>
                     case putAt t p new of
                       Nothing => MkFactB s ps ds False
                       Just t' => loop t' (S s) (ps + pst) (ds + dst)
                                      (inv && iv) fuel

-- ---------------------------------------------------------------
-- E4 : enumerative search over small representations
-- ---------------------------------------------------------------
```

```
sizedTerms : Nat -> SortedMap Nat (List Term)
sizedTerms top = foldl step SortedMap.empty (upTo 1 top)
  where
    step : SortedMap Nat (List Term) -> Nat -> SortedMap Nat (List Term)
    step m 1 = SortedMap.insert 1 [CK, CS] m
    step m n =
      let row = cmap (\k =>
                  let ls = fromMaybe [] (SortedMap.lookup k m)
                      rs = fromMaybe [] (SortedMap.lookup (minus n k) m)
                  in cmap (\a => map (App a) rs) ls)
                (upTo 1 (minus n 1))
      in SortedMap.insert n row m

enumUpTo : Nat -> List Term
enumUpTo s =
  let m = sizedTerms s
  in cmap (\i => fromMaybe [] (SortedMap.lookup i m)) (upTo 1 s)

wnfM : Nat -> Term -> Maybe Term
wnfM Z     _ = Nothing
wnfM (S f) t = case stepW t of
                 Nothing => Just t
                 Just t' => wnfM f t'

wnfStepsM : Nat -> Term -> Maybe (Term, Nat)
wnfStepsM fuel t = go fuel Z t
  where
    go : Nat -> Nat -> Term -> Maybe (Term, Nat)
    go Z     _ _ = Nothing
    go (S f) c u = case stepW u of
                     Nothing => Just (u, c)
                     Just u' => go f (S c) u'

decodeIter : Term -> Maybe Nat
decodeIter (Var v)         = if v == "x" then Just Z else Nothing
decodeIter (App (Var g) r) = if g == "f" then map S (decodeIter r) else Nothing
decodeIter _               = Nothing

E4FUEL : Nat
E4FUEL = 1200

E4VCAP : Nat
E4VCAP = 1000

E4WCAP : Nat
E4WCAP = 300

E4OFUEL : Nat
E4OFUEL = 6000

iterVal : Term -> Maybe Nat
iterVal t = wnfM E4FUEL (App (App t (Var "f")) (Var "x")) >>= decodeIter

iterValSteps : Term -> Maybe (Nat, Nat)
```

```
iterValSteps t = do
  (nf, st) <- wnfStepsM E4FUEL (App (App t (Var "f")) (Var "x"))
  v <- decodeIter nf
  pure (v, st)

||| orbit values from a seed; healthy if defined, within cap, all distinct
orbitFrom : Term -> Term -> Maybe (List Nat)
orbitFrom seed c = go seed 4 []
  where
    go : Term -> Nat -> List Nat -> Maybe (List Nat)
    go _   Z     acc = if length (nub acc) == 4 && all (<= E4VCAP) acc
                       then Just (reverse acc) else Nothing
    go cur (S k) acc =
      if tsize cur > 300 then Nothing
      else case wnfM E4OFUEL (App (App cur (Var "f")) (Var "x")) >>= decodeIter of
             Just v  => if v <= E4VCAP then go (App c cur) k (v :: acc) else
Nothing
             Nothing => Nothing

||| orbit from the base, or from base plus two for combination-only actions
orbitSig : Term -> Term -> Maybe (List Nat)
orbitSig z c =
  case orbitFrom z c of
    Just sig => Just (0 :: sig)
    Nothing  => map (2 ::) (orbitFrom (unary 2) c)

||| dedupe constructors by their action signature on the orbit
dedupeBySig : Term -> List Term -> List Term
dedupeBySig z cs = go cs SortedSet.empty
  where
    go : List Term -> SortedSet (List Nat) -> List Term
    go []        _    = []
    go (c :: rs) seen =
      case orbitSig z c of
        Nothing  => go rs seen
        Just sig => if SortedSet.contains sig seen
                    then go rs seen
                    else c :: go rs (SortedSet.insert sig seen)

||| all words over ops applied to z, lengths 0 .. maxL
wordsUpTo : Nat -> List Term -> Term -> List Term
wordsUpTo maxL ops z = go (S maxL) [z]
  where
    go : Nat -> List Term -> List Term
    go Z     _   = []
    go (S k) cur = cur ++ go k (cmap (\c => map (App c) cur) ops)

record VStat where
  constructor MkVS
  vsMinLenT : Term      -- a shortest word of this value
  vsCount   : Nat       -- how many words hit this value

||| Krivine-style weak head machine: O(1) per rule firing
whnfSt : Nat -> Term -> List Term -> Maybe (Term, List Term, Nat)
whnfSt Z     _   _  = Nothing
```

```
whnfSt (S k) (App f a) st              = whnfSt k f (a :: st)
whnfSt (S k) CK (x :: _ :: st)         = whnfSt k x st
whnfSt (S k) CS (a :: b :: c :: st)    = whnfSt k a (c :: App b c :: st)
whnfSt fuel  h   st                    = Just (h, st, fuel)

decodeSpine : Nat -> Term -> Maybe (Nat, Nat)
decodeSpine fuel t = do
  (h, st, f') <- whnfSt fuel t []
  case (h, st) of
    (Var "x", [])  => Just (0, f')
    (Var "f", [u]) => do
      (v, f'') <- decodeSpine f' u
      pure (S v, f'')
    _ => Nothing

iterFast : Nat -> Term -> Maybe Nat
iterFast fuel t =
  map fst (decodeSpine fuel (App (App t (Var "f")) (Var "x")))

valueTable : Nat -> List Term -> Term -> SortedMap Nat VStat
valueTable maxL ops z = foldl add SortedMap.empty (wordsUpTo maxL ops z)
  where
    add : SortedMap Nat VStat -> Term -> SortedMap Nat VStat
    add m w =
      if tsize w > 400 then m
      else case iterFast 1400 w of
        Nothing => m
        Just v  =>
          if v > E4WCAP then m
          else case SortedMap.lookup v m of
                 Nothing => SortedMap.insert v (MkVS w 1) m
                 Just s  => SortedMap.insert v (MkVS s.vsMinLenT (S s.vsCount)) m

fingerprint : Nat -> Term -> Nat            -- depth-bounded root shape code
fingerprint Z     CK        = 1
fingerprint Z     CS        = 2
fingerprint Z     (Var _)   = 3
fingerprint Z     (App _ _) = 4
fingerprint (S d) (App l r) = 5 + 7 * fingerprint d l + 97 * fingerprint d r
fingerprint (S d) t         = fingerprint Z t

isPrimeN : Nat -> Bool
isPrimeN v = v >= 2 && lpf v == v

||| exact separation of primes vs composites by threshold or category
separates : List Nat -> List Nat -> Bool
separates ps cs =
  let mxp = foldl max 0 ps
      mnp = foldl min 9999 ps
      mxc = foldl max 0 cs
      mnc = foldl min 9999 cs
      catP = mxp == mnp && not (elem mxp cs)
      catC = mxc == mnc && not (elem mxc ps)
  in not (null ps) && not (null cs)
     && (mxp < mnc || mxc < mnp || catP || catC)
```

```
record FRow where
  constructor MkFRow
  frMinLen : Nat
  frCount  : Nat
  frSteps  : Nat
  frNF     : Nat
  frFP     : Nat

frow : VStat -> FRow
frow s =
  let nf = wnfM E4FUEL s.vsMinLenT
  in MkFRow (tsize s.vsMinLenT)
            s.vsCount
            (fromMaybe 0 (map snd (iterValSteps s.vsMinLenT)))
            (fromMaybe 0 (map tsize nf))
            (fromMaybe 0 (map (fingerprint 2) nf))

record Screen where
  constructor MkScreen
  scCov   : Nat
  scOK    : Bool
  scFeats : List String

||| screen an alphabet over window 2 .. winTop: coverage plus separation
screen : Nat -> Nat -> List Term -> Term -> Screen
screen winTop maxL ops z =
  let tab   = valueTable maxL ops z
      rows  = mapMaybe (\v => map (\st => (v, frow st))
                                  (SortedMap.lookup v tab))
                       (upTo 2 winTop)
      prims = filter (\r => isPrimeN (fst r)) rows
      comps = filter (\r => not (isPrimeN (fst r))) rows
      cov   = length rows
      ok    = isJust (SortedMap.lookup 5 tab) && isJust (SortedMap.lookup 6 tab)
              && length prims >= 4 && length comps >= 4
      pv    = \g => map (g . snd) prims
      cv    = \g => map (g . snd) comps
      feats = filter (\nf => separates (pv (snd nf)) (cv (snd nf)))
                [ ("minLen", \r => r.frMinLen)
                , ("count",  \r => r.frCount)
                , ("steps",  \r => r.frSteps)
                , ("nfSize", \r => r.frNF)
                , ("rootFP", \r => r.frFP) ]
  in MkScreen cov ok (if ok then map fst feats else [])

pairsOf : List a -> List (List a)
pairsOf []        = []
pairsOf (x :: xs) = map (\y => [x, y]) xs ++ pairsOf xs

triplesOf : List a -> List (List a)
triplesOf []        = []
triplesOf (x :: xs) = map (\p => x :: p) (pairsOf xs) ++ triplesOf xs
```

```
-----------------------------------------------------------------
-- E4 at scale : normal-form enumeration, chunked scan, rank
-----------------------------------------------------------------

ser : Term -> String
ser CK        = "k"
ser CS        = "s"
ser (Var _)   = "v"
ser (App a b) = "a" ++ ser a ++ ser b

deser1 : List Char -> Maybe (Term, List Char)
deser1 ('k' :: r) = Just (CK, r)
deser1 ('s' :: r) = Just (CS, r)
deser1 ('a' :: r) = do
  (l, r1)  <- deser1 r
  (rt, r2) <- deser1 r1
  pure (App l rt, r2)
deser1 _ = Nothing

deser : String -> Maybe Term
deser str = case deser1 (unpack str) of
              Just (t, []) => Just t
              _            => Nothing

||| weak normalization with a fuel bound and a periodic size cap
wnfCapped : Nat -> Nat -> Term -> Maybe Term
wnfCapped cap fuel t0 = go fuel t0
  where
    go : Nat -> Term -> Maybe Term
    go Z     _ = Nothing
    go (S f) t =
      case stepW t of
        Nothing => Just t
        Just t' => if modN f 32 == 0 && tsize t' > cap
                     then Nothing
                     else go f t'

iterValC : Nat -> Nat -> Term -> Maybe Nat
iterValC fuel cap t =
  wnfCapped cap fuel (App (App t (Var "f")) (Var "x")) >>= decodeIter

||| phased orbit: cheap first application, generous deeper points
orbitB : Term -> Term -> Maybe (List Nat)
orbitB seed c = do
  v0 <- iterValC 600 1600 seed
  v1 <- iterValC 400 900 (App c seed)
  let t2 = if v1 > 60 || v1 == v0 then Var "dead" else App c (App c seed)
  v2 <- iterValC 5000 2500 t2
  let vs3 = [v0, v1, v2]
  if not (length (nub vs3) == 3 && all (<= E4VCAP) vs3)
    then Nothing
    else case iterValC 5000 2500 (App c t2) of
      Just v3 =>
        if v3 <= E4VCAP && length (nub (v3 :: vs3)) == 4
          then Just [v0, v1, v2, v3]
```

```
          else growing vs3
      Nothing => growing vs3
  where
    growing : List Nat -> Maybe (List Nat)
    growing [a, b, c2] = if a < b && b < c2 then Just [a, b, c2] else Nothing
    growing _          = Nothing


orbitFast : Term -> Term -> Maybe (List Nat)
orbitFast seed c = do
  v0 <- iterFast 2000 seed
  v1 <- iterFast 400 (App c seed)
  let t2 = if v1 > 60 || v1 == v0 then Var "dead" else App c (App c seed)
  v2 <- iterFast 1500 t2
  let vs3 = [v0, v1, v2]
  if not (length (nub vs3) == 3 && all (<= E4VCAP) vs3)
    then Nothing
    else case iterFast 1500 (App c t2) of
      Just v3 =>
        if v3 <= E4VCAP && length (nub (v3 :: vs3)) == 4
          then Just [v0, v1, v2, v3]
          else growingF vs3
      Nothing => growingF vs3
  where
    growingF : List Nat -> Maybe (List Nat)
    growingF [a, b, c2] = if a < b && b < c2 then Just [a, b, c2] else Nothing
    growingF _          = Nothing

sigFast : Term -> Maybe (List Nat)
sigFast (App CK _)           = Nothing   -- constant maps
sigFast (App (App CS CK) _)  = Nothing   -- S K x is the identity
sigFast (App (App CS CS) _)  = Nothing   -- S S x is never an iterator
sigFast c =
  case orbitFast iC c of
    Just sg => Just (0 :: sg)
    Nothing => map (2 ::) (orbitFast (unary 2) c)

sigB : Term -> Maybe (List Nat)
sigB (App CK _)          = Nothing
sigB (App (App CS CK) _) = Nothing
sigB (App (App CS CS) _) = Nothing
sigB c =
  case orbitB iC c of
    Just sg => Just (0 :: sg)
    Nothing => map (2 ::) (orbitB (unary 2) c)

||| closed weak normal forms by leaf count: N ::= K | S | K N | S N | S N N
nfTable : Nat -> SortedMap Nat (List Term)
nfTable top = foldl step SortedMap.empty (upTo 1 top)
  where
    step : SortedMap Nat (List Term) -> Nat -> SortedMap Nat (List Term)
    step m 1 = SortedMap.insert 1 [CK, CS] m
    step m n =
      let prev = fromMaybe [] (SortedMap.lookup (minus n 1) m)
          uns  = map (App CK) prev ++ map (App CS) prev
```

```
          bins = cmap (\a =>
                   let as = fromMaybe [] (SortedMap.lookup a m)
                       bs = fromMaybe [] (SortedMap.lookup (minus (minus n 1) a) m)
                   in cmap (\x => map (App (App CS x)) bs) as)
                 (upTo 1 (minus n 2))
      in SortedMap.insert n (uns ++ bins) m

scanLoop : SortedSet (List Nat) -> Nat -> Nat -> List Term -> IO Nat
scanLoop _    _    kept []        = pure kept
scanLoop seen done kept (c :: cs) = do
  when (modN done 20000 == 0 && done > 0) $
    ignore (fPutStrLn stderr ("  .. " ++ show done ++ " scanned, " ++ show kept ++
" kept"))
  case sigFast c of
    Nothing => scanLoop seen (S done) kept cs
    Just sg =>
      if SortedSet.contains sg seen
        then scanLoop seen (S done) kept cs
        else do
          ignore (appendFile "/home/claude/e4keep.txt" (ser c ++ "\n"))
          scanLoop (SortedSet.insert sg seen) (S done) (S kept) cs

nfPartT : SortedMap Nat (List Term) -> Nat -> Nat -> List Term
nfPartT tab siz p =
  if p == 0 then map (App CK) (row (minus siz 1))
  else if p == 1 then map (App CS) (row (minus siz 1))
  else let a = minus p 1
       in cmap (\x => map (App (App CS x)) (row (minus (minus siz 1) a)))
                       (row a)
  where
    row : Nat -> List Term
    row i = fromMaybe [] (SortedMap.lookup i tab)

scanOne : String -> List Term -> IO ()
scanOne label pool = do
  kept <- scanLoop SortedSet.empty Z Z pool
  ignore (fPutStrLn stderr (label ++ ": scanned " ++ show (length pool)
           ++ ", kept " ++ show kept))

nfPart : Nat -> Nat -> List Term
nfPart siz p =
  let m = nfTable (minus siz 1)
      nf = \i => fromMaybe [] (SortedMap.lookup i m)
  in if p == 0 then map (App CK) (nf (minus siz 1))
     else if p == 1 then map (App CS) (nf (minus siz 1))
     else let a = minus p 1
          in cmap (\x => map (App (App CS x)) (nf (minus (minus siz 1) a)))
                          (nf a)

e4scan : List String -> IO ()
e4scan ["dbg", sStr, pStr, nStr] = do
  let siz = fromMaybe 1 (parsePositive sStr)
  let prt = fromMaybe 0 (parsePositive pStr)
  let n   = fromMaybe 1 (parsePositive nStr)
  ignore (appendFile "/tmp/dbg.txt" "start\n")
```

```
  let pool = take n (nfPart siz prt)
  ignore (appendFile "/tmp/dbg.txt" ("pool ready: " ++ show (length pool) ++ "\n"))
  traverse_ (\(i, c) => do
      ignore (appendFile "/tmp/dbg.txt" (show i ++ " " ++ ser c ++ "\n"))
      ignore (appendFile "/tmp/dbg.txt" ("  -> " ++ show (sigFast c) ++ "\n")))
    (zip (upTo 0 (minus n 1)) pool)
e4scan ["nfr", sStr, loStr, hiStr] = do
  let siz = fromMaybe 1 (parsePositive sStr)
  let lo  = fromMaybe 1 (parsePositive loStr)
  let hi  = fromMaybe 1 (parsePositive hiStr)
  let tab = nfTable (minus siz 1)
  traverse_ (\p => scanOne ("nfr " ++ show siz ++ " part " ++ show p)
                           (nfPartT tab siz p)) (upTo lo hi)
e4scan ["nfr2", sStr, loStr, hiStr] = do
  let siz = fromMaybe 1 (parsePositive sStr)
  let lo  = fromMaybe 1 (parsePositive loStr)
  let hi  = fromMaybe 1 (parsePositive hiStr)
  let tab = nfTable (minus siz 2)
  traverse_ (\p => scanOne ("nfr2 " ++ show siz ++ " part " ++ show p)
                           (nfPartT tab siz p)) (upTo lo hi)
e4scan ["n15p1a", loS, hiS] = do
  let tab = nfTable 12
  traverse_ (\q => scanOne ("n15p1a q " ++ show q)
              (map (\x => App CS (App CK x)) (nfPartT tab 13 q)))
            (upTo (fromMaybe 0 (parsePositive loS)) (fromMaybe 0 (parsePositive
hiS)))
e4scan ["n15p1b", loS, hiS] = do
  let tab = nfTable 12
  traverse_ (\q => scanOne ("n15p1b q " ++ show q)
              (map (\x => App CS (App CS x)) (nfPartT tab 13 q)))
            (upTo (fromMaybe 0 (parsePositive loS)) (fromMaybe 0 (parsePositive
hiS)))
e4scan ["n15p1c", loS, hiS] = do
  let tab = nfTable 12
  traverse_ (\q => scanOne ("n15p1c q " ++ show q)
              (map (App CS) (nfPartT tab 14 q)))
            (upTo (fromMaybe 0 (parsePositive loS)) (fromMaybe 0 (parsePositive
hiS)))
e4scan ["n15p14", loS, hiS] = do
  let tab = nfTable 12
  traverse_ (\q => scanOne ("n15p14 q " ++ show q)
              (cmap (\x => [App (App CS x) CK, App (App CS x) CS]) (nfPartT tab 13
q)))
            (upTo (fromMaybe 0 (parsePositive loS)) (fromMaybe 0 (parsePositive
hiS)))
e4scan ["nft", sStr, dStr, loS, hiS] = do
  let siz = fromMaybe 1 (parsePositive sStr)
  let tab = nfTable (fromMaybe 1 (parsePositive dStr))
  traverse_ (\p => scanOne ("nft " ++ show siz ++ " part " ++ show p)
                           (nfPartT tab siz p))
            (upTo (fromMaybe 1 (parsePositive loS)) (fromMaybe 1 (parsePositive
hiS)))
e4scan ["nqr", sStr] = do
  let siz = fromMaybe 1 (parsePositive sStr)
  let tab = nfTable (minus siz 2)
```

```
  traverse_ (\q => scanOne ("nqr " ++ show siz ++ " q " ++ show q)
                           (map (App CS) (nfPartT tab (minus siz 1) q)))
            (upTo 0 (minus siz 2))
e4scan ["nfp", sStr, pStr, iStr, mStr] = do
  let siz = fromMaybe 1 (parsePositive sStr)
  let prt = fromMaybe 0 (parsePositive pStr)
  let idx = fromMaybe 0 (parsePositive iStr)
  let m   = fromMaybe 1 (parsePositive mStr)
  let pool = nfPart siz prt
  let len   = length pool
  let per   = divN (minus (len + m) 1) m
  let slice = take per (drop (idx * per) pool)
  kept <- scanLoop SortedSet.empty Z Z slice
  putStrLn $ "nfp " ++ show siz ++ " part " ++ show prt ++ " chunk "
           ++ show idx ++ "/" ++ show m ++ " of " ++ show len
           ++ ": scanned " ++ show (length slice) ++ ", kept " ++ show kept
e4scan [mode, sStr, iStr, mStr] = do
  let siz = fromMaybe 1 (parsePositive sStr)
  let idx = fromMaybe 0 (parsePositive iStr)
  let m   = fromMaybe 1 (parsePositive mStr)
  let pool = if mode == "nf"
               then fromMaybe [] (SortedMap.lookup siz (nfTable siz))
               else fromMaybe [] (SortedMap.lookup siz (sizedTerms siz))
  let len   = length pool
  ignore (appendFile "/home/claude/e4prog.txt" ("pool ready: " ++ show len ++
"\n"))
  let per   = divN (minus (len + m) 1) m
  let slice = take per (drop (idx * per) pool)
  kept <- scanLoop SortedSet.empty Z Z slice
  ignore (appendFile "/home/claude/e4prog.txt"
            ("DONE " ++ mode ++ " " ++ show siz ++ " " ++ show idx ++ "/" ++ show m
             ++ " kept " ++ show kept ++ "\n"))
  putStrLn $ mode ++ " size " ++ show siz ++ " chunk " ++ show idx ++ "/"
           ++ show m ++ " of " ++ show len ++ ": scanned "
           ++ show (length slice) ++ ", kept " ++ show kept

e4scan _ = putStrLn "usage: e4scan (nf|raw) size chunk chunks"

dedupSmallest : List (List Nat, Term) -> List (List Nat, Term)
dedupSmallest = foldl ins []
  where
    ins : List (List Nat, Term) -> (List Nat, Term) -> List (List Nat, Term)
    ins acc (sg, c) =
      case lookup sg acc of
        Nothing => (sg, c) :: acc
        Just c0 => if tsize c < tsize c0
                     then (sg, c) :: filter (\p => fst p /= sg) acc
                     else acc

e4rank : IO ()
e4rank = do
  Right txt <- readFile "/home/claude/e4keep.txt"
    | Left _ => putStrLn "no keeper file"
  let cands   = mapMaybe deser (filter (/= "") (lines txt))
  let withSig = mapMaybe (\c => map (\sg => (sg, c)) (sigB c)) cands
```

```
  let acts0   = dedupSmallest withSig
  let acts    = sortBy (\p, q => compare (tsize (snd p)) (tsize (snd q))) acts0
  putStrLn $ "== E4 rank: " ++ show (length cands) ++ " keepers, "
           ++ show (length acts) ++ " distinct actions =="
  traverse_ (\(sg, c) => putStrLn $ "  size " ++ show (tsize c) ++ "  "
                       ++ show c ++ "  orbit " ++ show sg) acts
  let ops  = map snd (take 40 acts)
  let opsT = take 12 ops
  let alph = map (\c => [c]) ops ++ pairsOf ops ++ triplesOf opsT
  putStrLn $ "  alphabets over these actions: " ++ show (length alph)
  let scored  = map (\o => (o, screen 10 (if length o == 1 then 12
                                          else if length o == 2 then 8 else 7) o
iC)) alph
  let covered = filter (\oc => (snd oc).scOK) scored
  let promis  = filter (\oc => not (null ((snd oc).scFeats))) covered
  putStrLn $ "  stage 1 (window 2..10, 5 and 6 required): "
           ++ show (length covered) ++ " cover, "
           ++ show (length promis) ++ " promising"
  traverse_ (\(o, sc) => do
      let s2 = screen 30 (if length o == 1 then 32 else 11) o iC
      putStrLn $ "  promising: { "
               ++ concat (intersperse " , " (map show o))
               ++ " } via " ++ concat (intersperse "," sc.scFeats)
               ++ "  -> stage 2 (window 2..30): "
               ++ (if null s2.scFeats
                     then "eliminated"
                     else "SURVIVOR via " ++ concat (intersperse "," s2.scFeats)))
    promis


-----------------------------------------------------------------------
-- E5 : residue bundles, the K/S-native number system
-----------------------------------------------------------------------

pairT : Term -> Term -> Term             -- S(S(K a))(K b), inert two-slot carrier
pairT a b = App (App CS (App CS (App CK a))) (App CK b)

unpairT : Term -> Maybe (Term, Term)
unpairT (App (App CS (App CS (App CK a))) (App CK b)) = Just (a, b)
unpairT _ = Nothing

basisA : List Nat
basisA = [2, 3, 5, 7]                   -- modulus 210, primality window below 121

basisB : List Nat
basisB = [2, 3, 5, 7, 11, 13]           -- modulus 30030, window below 289

bundleOf : List Nat -> Nat -> Term
bundleOf [p]       n = unary (modN n p)
bundleOf (p :: ps) n = pairT (unary (modN n p)) (bundleOf ps n)
bundleOf []        _ = ki

bundleRes : Term -> Maybe (List Nat)
bundleRes t =
  case unpairT t of
```

```
    Just (a, rest) => do
      r  <- combLen a
      rs <- bundleRes rest
      pure (r :: rs)
    Nothing => map (\r => [r]) (combLen t)

coordAt : Nat -> Term -> Maybe Term
coordAt Z     t = case unpairT t of
                    Just (a, _) => Just a
                    Nothing     => Just t
coordAt (S i) t = unpairT t >>= coordAt i . snd

||| divisibility by the i-th basis prime: a window read at depth i
divisAt : Nat -> Term -> Maybe Bool
divisAt i t = map (== ki) (coordAt i t)

||| coordinatewise rule tables: one local rewrite instance per coordinate
opB : (Nat -> Nat -> Nat -> Nat) -> List Nat -> Term -> Term -> Maybe (Term, Nat)
opB f basis x y = do
  rx <- bundleRes x
  ry <- bundleRes y
  if length rx == length basis && length ry == length basis
    then let rs = zipWith3 f basis rx ry
         in Just (rebuild basis rs, length basis)
    else Nothing
  where
    zipWith3 : (Nat -> Nat -> Nat -> Nat) -> List Nat -> List Nat -> List Nat ->
List Nat
    zipWith3 g (p :: ps) (a :: as) (b :: bs) = g p a b :: zipWith3 g ps as bs
    zipWith3 _ _ _ _ = []
    rebuild : List Nat -> List Nat -> Term
    rebuild [_]       [r]       = unary r
    rebuild (_ :: ps) (r :: rs) = pairT (unary r) (rebuild ps rs)
    rebuild _ _                 = ki

addB : List Nat -> Term -> Term -> Maybe (Term, Nat)
addB = opB (\p, a, b => modN (a + b) p)

mulB : List Nat -> Term -> Term -> Maybe (Term, Nat)
mulB = opB (\p, a, b => modN (a * b) p)

succB : List Nat -> Term -> Maybe (Term, Nat)
succB basis t = addB basis t (bundleOf basis 1)

||| primality on the validity window: in the basis, or all coordinates nonzero
primeByBundle : List Nat -> Nat -> Term -> Bool
primeByBundle basis n t =
  elem n basis || fromMaybe False (map (all (/= 0)) (bundleRes t))

||| encoder composed from the E2 probes: one per basis prime
encodeB : List Nat -> Nat -> (Term, Nat, Bool)
encodeB basis n =
  let ps  = map (\p => runProbe p n) basis
      rs  = map (\pr => pr.pResidue) ps
      st  = sum (map (\pr => pr.pSteps) ps)
```

```
      ok  = all (\pr => pr.pInvOK) ps
      bld = go basis rs
  in (bld, st, ok)
  where
    go : List Nat -> List Nat -> Term
    go [_]       [r]       = unary r
    go (_ :: ps) (r :: rs) = pairT (unary r) (go ps rs)
    go _ _                 = ki

iterSucc : List Nat -> Nat -> Term -> Term
iterSucc _     Z     t = t
iterSucc basis (S k) t = case succB basis t of
                           Just (t', _) => iterSucc basis k t'
                           Nothing      => t
--------------------------------------------------------------------
-- main
--------------------------------------------------------------------

suite : IO ()
suite = do
  putStrLn "== E1: binary numerals over doubling and successor =="
  putStrLn $ "  bin 1  = " ++ show (bin 1)
  putStrLn $ "  bin 6  = " ++ show (bin 6)

  putStrLn "\n== adequacy: bin n f x =w unary n f x, n = 1 .. 24 =="
  putStrLn $ "  all agree: " ++ boolMark (all adequate (upTo 1 24))

  putStrLn "\n== parity is root-local, n = 1 .. 64 =="
  putStrLn $ "  root window decides n mod 2: "
           ++ boolMark (all parityLocal (upTo 1 64))

  putStrLn "\n== divisibility by 4 is depth-2-local, even n = 2 .. 64 =="
  putStrLn $ "  depth-2 window decides n mod 4: "
           ++ boolMark (all mod4Local (filter (\n => modN n 2 == 0) (upTo 2 64)))

  putStrLn "\n== size census: tree size vs distinct subterms (DAG size) =="
  traverse_ (\n =>
      let t = bin n
      in putStrLn $ "  n=" ++ show n ++ ": tree " ++ show (tsize t)
                 ++ ", distinct " ++ show (countDistinct t))
    [15, 16, 255, 256, 1023, 1024]

  putStrLn "\n== the frozen product persists for binary numerals =="
  putStrLn $ "  wNF(B bin3 bin5) == S(K bin3)bin5: "
           ++ boolMark (wnf (multB (bin 3) (bin 5)) == prodBox (bin 3) (bin 5))

  putStrLn "\n== the doubling combinator, derived by abstraction =="
  putStrLn $ "  D = " ++ show dComb
  putStrLn $ "  wNF(D bin3) == bin 6: "
           ++ boolMark (wnf (App dComb (bin 3)) == bin 6)

  putStrLn "\n== orientation conflict: R_arith flattens binary to unary =="
  traverse_ (\n =>
      putStrLn $ "  nfR(bin " ++ show n ++ ") == unary " ++ show n ++ ": "
               ++ boolMark (nfR (bin n) == unary n))
```

```
    [6, 7, 12]

  putStrLn "\n== E2: divisibility probes as local rewrites =="
  let sample = mkMark (unary 2) (unary 1) (bin 5)
  putStrLn $ "  probe wrapper is inert under weak reduction: "
           ++ boolMark (stepW sample == Nothing)
  putStrLn $ "  wrapper adds no R_arith redex (unary payload): "
           ++ boolMark (stepR (mkMark (unary 2) (unary 1) (unary 5)) == Nothing)
  putStrLn $ "  R_arith still flattens a binary payload (the orientation conflict):
"
           ++ boolMark (isJust (stepR sample))

  putStrLn "\n== E2 trace: n = 13, k = 3 =="
  let tr = runProbe 3 13
  putStrLn $ "   " ++ concat (intersperse "  ~>  " tr.pTrace)
  putStrLn $ "  residue " ++ show tr.pResidue ++ " in " ++ show tr.pSteps
           ++ " steps, invariant: " ++ boolMark tr.pInvOK

  putStrLn "\n== E2 sweep: n = 2 .. 40, k = 2 .. n =="
  let ok = all (\(n, k) =>
            let pr = runProbe k n
            in pr.pResidue == modN n k && pr.pInvOK && pr.pSteps == bitLen n)
           (cmap (\n => map (\k => (n, k)) (upTo 2 n)) (upTo 2 40))
  putStrLn $ "  residue == n mod k, invariant holds, steps == bitlen n: "
           ++ boolMark ok

  putStrLn "\n== E2 residue vectors vs the unary strides of secret1 =="
  traverse_ (\(n, vec1, st1) =>
      let (vec2, st2, inv) = residueVecE2 n
      in putStrLn $ "  n=" ++ show n ++ ": vectors match: "
                 ++ boolMark (vec1 == vec2 && inv)
                 ++ ", steps " ++ show st1 ++ " unary -> " ++ show st2 ++ "
binary")
    [ (6,  [0, 0, 2, 1], 35)
    , (7,  [1, 1, 3, 2, 1], 35)
    , (11, [1, 2, 3, 1, 5, 4, 3, 2, 1], 94)
    , (12, [0, 0, 0, 2, 0, 5, 4, 3, 2, 1], 144)
    , (13, [1, 1, 1, 3, 1, 6, 5, 4, 3, 2, 1], 138)
    , (15, [1, 0, 3, 0, 3, 1, 7, 6, 5, 4, 3, 2, 1], 198) ]

  putStrLn "\n== E2 primality certificates, probes for k = 2 .. isqrt n =="
  traverse_ (\n =>
      case certify n of
        (True, _, np, st) =>
          putStrLn $ "  n=" ++ show n ++ ": PRIME  (" ++ show np
                   ++ " probes, " ++ show st ++ " steps)"
        (False, mf, np, st) =>
          putStrLn $ "  n=" ++ show n ++ ": composite, smallest factor "
                   ++ show (fromMaybe 0 mf) ++ "  (" ++ show np
                   ++ " probes, " ++ show st ++ " steps)")
    [91, 97, 143, 1001, 1009, 8191]

  putStrLn "\n== E3: F_arith, canonicalization rules are pointwise valid =="
  putStrLn $ "  Fswap  3*2 = 2*3: "
           ++ boolMark (pointEq (prodBox (unary 3) (unary 2))
```

```
                              (prodBox (unary 2) (unary 3)))
  putStrLn $ "  Fassoc (2*3)*5 = 2*(3*5): "
           ++ boolMark (pointEq (prodBox (prodBox (unary 2) (unary 3)) (unary 5))
                              (prodBox (unary 2) (prodBox (unary 3) (unary 5))))
  putStrLn $ "  Fsort  3*(2*5) = 2*(3*5): "
           ++ boolMark (pointEq (prodBox (unary 3) (prodBox (unary 2) (unary 5)))
                              (prodBox (unary 2) (prodBox (unary 3) (unary 5))))

  putStrLn "\n== E3: primality is F-irreducibility, n = 2 .. 200 =="
  putStrLn $ "  no split move exists  <=>  n prime: "
           ++ boolMark (all (\n => null (splitMoves (unary n)) == (lpf n == n))
                            (upTo 2 200))

  putStrLn "\n== E3: probe-guided certified factorization =="
  traverse_ (\n =>
      let r = factorPipeline n
      in putStrLn $ "  " ++ show n ++ " = " ++ showFact n
                 ++ "  (" ++ show r.frSplits ++ " splits, probes "
                 ++ show r.frProbe ++ ", certification "
                 ++ show r.frCert ++ " R-steps, canonical: "
                 ++ boolMark r.frOK ++ ")")
    [30, 97, 128, 143, 210, 1001, 1024]

  putStrLn "\n== E3: confluence = unique factorization =="
  putStrLn $ "  all first splits of all composite n <= 100 rejoin at canonF: "
           ++ boolMark (all firstSplitsConfluent
                            (filter (\n => lpf n /= n) (upTo 4 100)))
  putStrLn $ "  random full derivations, seeds 1..3, n <= 100: "
           ++ boolMark (all (\n => all (\s => randRun s n) [1, 2, 3])
                            (upTo 2 100))
  putStrLn $ "  branching at the root of 60 (ordered proper splits): "
           ++ show (length (splitMoves (unary 60)))

  putStrLn "\n== E3.2: rewrite division on binary leaves =="
  let d = runDiv 7 1001 (bin 1001)
  putStrLn $ "  1001 / 7: quotient == bin 143: " ++ boolMark (d.dQ == bin 143)
           ++ ", remainder " ++ show d.dRem ++ ", " ++ show d.dSteps
           ++ " transitions, invariant: " ++ boolMark d.dInvOK
  let d7 = runDiv 3 7 (bin 7)
  putStrLn $ "  3 does not divide 7: refused with remainder /= 0: "
           ++ boolMark (d7.dRem /= 0 && d7.dInvOK)

  putStrLn "\n== E3.2: certification cost, unary forward runs vs binary division
==="
  traverse_ (\(n, doUnary) =>
      let b = factorPipelineB n
          uc = if doUnary
                 then let u = factorPipeline n
                      in show u.frCert ++ (if u.frOK then "" else "?")
                 else "-"
      in putStrLn $ "  " ++ show n ++ " = " ++ showFact n
                 ++ ": unary " ++ uc
                 ++ " -> binary " ++ show b.fbDiv ++ " division steps ("
                 ++ show b.fbSplits ++ " splits, probes " ++ show b.fbProbe
                 ++ ", canonical: " ++ boolMark b.fbOK ++ ")")
```

```
    [ (30, True), (143, True), (210, True), (1001, True)
    , (1024, True), (4095, True), (65535, False) ]

  putStrLn "\n== E3.2: a large prime certificate via probes =="
  case certify 65537 of
    (True, _, np, st) =>
      putStrLn $ "  65537: PRIME  (" ++ show np ++ " probes, "
                 ++ show st ++ " steps)"
    (False, mf, np, st) =>
      putStrLn $ "  65537: composite, factor " ++ show (fromMaybe 0 mf)
                 ++ "  (" ++ show np ++ " probes, " ++ show st ++ " steps)"

  putStrLn "\n== E5: residue bundles, the K/S-native number system =="
  let bA = bundleOf basisA
  putStrLn $ "  bundle of 97 (basis 2,3,5,7): residues "
           ++ show (fromMaybe [] (bundleRes (bA 97)))
           ++ ", tree size " ++ show (tsize (bA 97))
  putStrLn $ "  carrier is inert under w and R_arith: "
           ++ boolMark (stepW (bA 97) == Nothing && stepR (bA 97) == Nothing)
  putStrLn $ "  fusion visible: pair a b X reduces to a (X b): "
           ++ boolMark (wnf (App (pairT (Var "a") (Var "b")) (Var "X"))
                       == App (Var "a") (App (Var "X") (Var "b")))
  putStrLn $ "  divisibility is a window read, n = 0..209, all basis primes: "
           ++ boolMark (all (\n => all (\i =>
                 divisAt i (bA n)
                   == Just (modN n (fromMaybe 2 (getAt' i basisA)) == 0))
                 (upTo 0 3)) (upTo 0 209))
  putStrLn $ "  primality is window reads, 2 <= n < 121: "
           ++ boolMark (all (\n => primeByBundle basisA n (bA n) == isPrimeN n)
                            (upTo 2 120))
  putStrLn $ "  six-prime basis, window 2 <= n < 289: "
           ++ boolMark (all (\n => primeByBundle basisB n (bundleOf basisB n)
                                     == isPrimeN n)
                            (upTo 2 288))
  putStrLn $ "  coordinatewise add and mul, all n, m <= 59, four local steps each:
"
           ++ boolMark (all (\(n, m) =>
                 addB basisA (bA n) (bA m) == Just (bA (modN (n + m) 210), 4)
                 && mulB basisA (bA n) (bA m) == Just (bA (modN (n * m) 210), 4))
               (cmap (\n => map (\m => (n, m)) (upTo 0 59)) (upTo 0 59)))
  let (e8191, st8191, inv8191) = encodeB basisB 8191
  putStrLn $ "  encoder from E2 probes: bundle of 8191 in " ++ show st8191
           ++ " local steps: "
           ++ boolMark (e8191 == bundleOf basisB 8191 && inv8191)
  putStrLn $ "  the paid costs arrive on schedule:"
  putStrLn $ "    aliasing: succ iterated 211 times from zero equals bundle of 1: "
           ++ boolMark (iterSucc basisA 211 (bA 0) == bA 1)
  putStrLn $ "    aliasing: bundle 6 times bundle 35 is the zero bundle: "
           ++ boolMark (map fst (mulB basisA (bA 6) (bA 35)) == Just (bA 0))

  putStrLn "\n== E4: enumerative search over small representations =="
  let allZ = enumUpTo 4
  let bases0 = filter (\z => iterVal z == Just 0) allZ
  let bases1 = filter (\z => iterVal z == Just 1) allZ
  putStrLn $ "  closed terms up to size 4: " ++ show (length allZ)
```

```
          ++ ", iterators of 0: " ++ show (length bases0)
          ++ ", of 1: " ++ show (length bases1)
  let clean1 = filter (\z => wnfM 200 (App z (Var "f")) == Just (Var "f")) bases1
  let zs = take 1 bases0 ++ take 1 (clean1 ++ bases1)
  putStrLn $ "  chosen bases: " ++ concat (intersperse ", " (map show zs))
  putStrLn $ "  positive control, the E3 box grammar separates n <= 30 exactly: "
          ++ boolMark (all (\n => null (splitMoves (unary n)) == isPrimeN n)
                           (upTo 2 30))
  traverse_ (\z => do
      let cs = dedupeBySig z (enumUpTo 8)
      let alph = map (\c => [c]) cs ++ pairsOf cs
      putStrLn $ "  base " ++ show z ++ ": "
               ++ show (length cs) ++ " constructor actions, "
               ++ show (length alph) ++ " alphabets"
      traverse_ (\c => putStrLn $ "    action " ++ show c ++ "  orbit "
                    ++ show (fromMaybe [] (orbitSig z c)))
        (take 12 cs)
      let scored = map (\ops =>
                    (ops, screen 10 (if length ops == 1 then 12 else 8) ops z))
                   alph
      let covered = filter (\oc => (snd oc).scOK) scored
      let promis  = filter (\oc => not (null ((snd oc).scFeats))) covered
      putStrLn $ "    stage 1 (window 2..10, with 5 and 6 required): "
               ++ show (length covered) ++ " cover, "
               ++ show (length promis) ++ " promising"
      traverse_ (\(ops, sc) => do
          let s2 = screen 30 (if length ops == 1 then 32 else 11) ops z
          putStrLn $ "    promising: { "
                   ++ concat (intersperse " , " (map show ops))
                   ++ " } via " ++ concat (intersperse "," sc.scFeats)
                   ++ "  -> stage 2 (window 2..30): "
                   ++ (if null s2.scFeats
                         then "eliminated"
                         else "SURVIVOR via " ++ concat (intersperse ","
s2.scFeats)))
          promis)
    zs

main : IO ()
main = do
  args <- getArgs
  case args of
    (_ :: "e4scan" :: rest) => e4scan rest
    (_ :: "e4rank" :: _)    => e4rank
    _                       => suite
```

Appendix C: The proof module, synchronized

The proof module Proofs.idr is a single artifact shared by every paper of this series. At this paper’s first verification pass it stood at 940 lines and was reproduced here verbatim; it now stands at 3,337 lines across parts A to X, still %default total, still without postulates or holes, and still verified in full by the one command idris2 —check Proofs.idr. Reproducing it verbatim in every paper stopped scaling and stopped being honest, since a copy is stale the day after it is made; from this version on, the code of record is the module itself, which accompanies the series, and this appendix carries its map and this paper’s anchors.

The module map, part banners with source lines: tier one, the shared calculus and its runs, lines 1 to 501; part A: R_arith as a relation, and fragment Theorem B, line 502; part B: the finite-exhaustion bridge, line 722; part C: split termination packaged as accessibility, line 760; part D: the divisibility probe, divmod correctness, line 778; part E: the linearity ladder (E8), line 937; part F: the soup absorber (E6), line 1041; part G: the circle and the octave (book chapters), line 1142; part H: the conserving chemistry balances (E6.2), line 1181; part I: the logic side (book chapters), line 1247; part J: the thrush (book chapter, the garden), line 1319; part K: agenda seeds (additive theorem B, cost of a box), line 1345; part L: no bird sings Peirce (L2), line 1403; part M: the workshop (book chapter two), line 1507; part N: the vanishing engine, verified (book chapter two), line 1541; part O: the coin preserved (L1, first half), line 1596; part P: the entropy budget (N4 weights, P1 reading), line 1659; part Q: the quoted alphabet (C5 stage one), line 1776; part R: the reader's engine (C5 stage two), line 1864; part S: every typed song ends (L1, second half), line 2051; part T: the first eye (C5 stage three, movement one), line 2230; part U: the second eye and the assembler (C5, movement two), line 2428; part V: two refusals, one ledger (occurrence conservation), line 2728; part W: the dispatcher (C5 stage three, movement three), line 2863; part X: the knot (C5 stage three, movement four), line 2983.

Anchors this paper relies on: part A carries R_arith as a relation; part B the finite-exhaustion bridge behind the census and the null separation; part C split termination as accessibility; part D the divisibility probe with divmod correctness and the certified witnesses (probeCert); part K the additive fragment and the box cost boxLen with rLenSum.

Developments since v0.8. The entropy budget (module part P, its own paper) weighs this paper's rewriting system directly: dupMass and eraseMass meter rule R1' s duplication and rule R2' s erasure along any run, boxDup and boxErase give the multiplication box its exact bill, and the Landauer reading turns the step-count asymmetry of Section 6 into a heat asymmetry, the fast order the hot order. The occurrence ledger of the companion linearity paper has since grown a quantum shadow, the two refusals with their bills, over the same two rules. And the reader campaign (parts Q to X) delivered a verified self-reader, so the automata of this paper are now themselves readable objects inside the calculus, with compiled agreement between the internal reader and the reference engine.

References

1. Project secret1, 'A Tree-Based Number System over the Combinators K and S, and Primality as Unreachability', working paper v0.4, August 11, 2026.
2. K. Bimbó, *Combinatory Logic: Pure, Applied and Typed*, CRC Press, 2012.
3. F. Baader and T. Nipkow, *Term Rewriting and All That*, Cambridge University Press, 1998.
4. M. Minsky and S. Papert, 'Unrecognizable sets of numbers', *Journal of the ACM* 13 (1966), 281-286.
5. J. Hartmanis and H. Shank, 'On the recognition of primes by automata', *Journal of the ACM* 15 (1968), 382-389.
6. J. Shallit, 'Automaticity IV: sequences, sets, and diversity', *Journal de Théorie des Nombres de Bordeaux* 8 (1996), 347-367.
7. M. Agrawal, N. Kayal, and N. Saxena, 'PRIMES is in P', *Annals of Mathematics* 160 (2004), 781-793.
8. P. L. Montgomery, 'Modular multiplication without trial division', *Mathematics of Computation* 44 (1985), 519-521.
9. D. E. Knuth, *The Art of Computer Programming, Volume 2: Seminumerical Algorithms*, third edition, Addison-Wesley, 1997.
10. P. Tarau, work on tree-represented natural numbers and their arithmetic, including hereditarily binary number systems (2013 onward).

11. E. Brady, ‘Idris 2: Quantitative Type Theory in Practice’, *Proceedings of ECOOP 2021*, LIPIcs 194, 2021.